

Module 3 - Core Java

NAME : MANOSHANKARI V

SUPERSET : 6736702

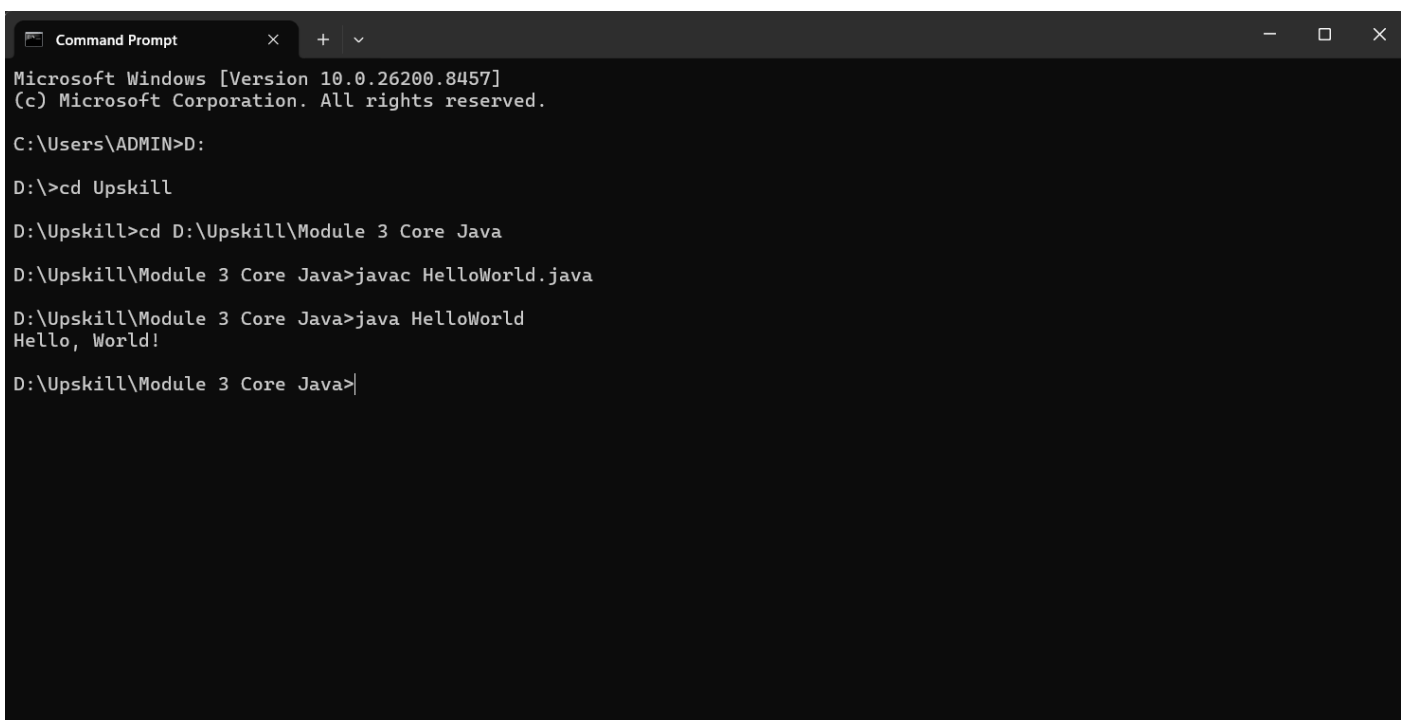
1.Hello World Program

Objective: Understand the basic structure of a Java program, including classes, the main() method, and console output.

Code:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Hello, World!");  
  
    }  
  
}
```

Output:

A screenshot of a Windows Command Prompt window. The title bar shows 'Command Prompt' with standard window controls. The text inside the window shows the following sequence of commands and output:
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ADMIN>D:
D:\>cd Upskill
D:\Upskill>cd D:\Upskill\Module 3 Core Java
D:\Upskill\Module 3 Core Java>javac HelloWorld.java
D:\Upskill\Module 3 Core Java>java HelloWorld
Hello, World!
D:\Upskill\Module 3 Core Java>|
The output 'Hello, World!' is displayed on the line following the 'java HelloWorld' command.

2. Simple Calculator

Objective: Practice arithmetic operations and handling user input.

Code:

```
import java.util.Scanner;

class Calculator{

    public static void main(String[] args) {

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter first number: ");

        double num1=sc.nextDouble();

        System.out.print("Enter second number: ");

        double num2=sc.nextDouble();

        System.out.print("Enter operator (+, -, *, /): ");

        char operator=sc.next().charAt(0);

        double ans=0;

        boolean valid=true;

        switch(operator){

            case '+':

                ans=num1+num2;

                break;

            case '-':

                ans=num1-num2;

                break;

            case '*':

                ans=num1*num2;

                break;

            case '/':

                ans=num1/num2;

                break;

            default:

                System.out.println("Invalid operator!");

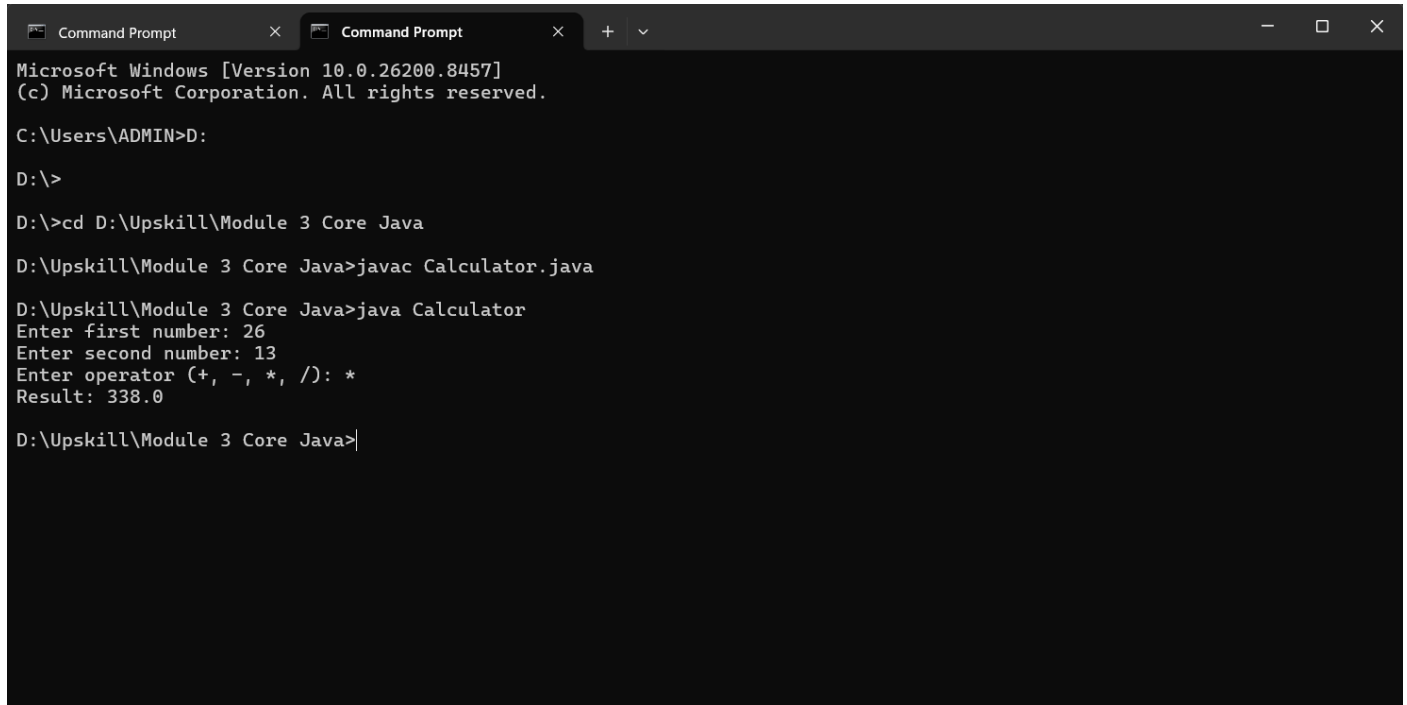
                valid=false;

        }

        if(valid){
```

```
        System.out.println("Result: "+ans);  
    }  
}  
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>  
  
D:\>cd D:\Upskill\Module 3 Core Java  
  
D:\Upskill\Module 3 Core Java>javac Calculator.java  
  
D:\Upskill\Module 3 Core Java>java Calculator  
Enter first number: 26  
Enter second number: 13  
Enter operator (+, -, *, /): *  
Result: 338.0  
  
D:\Upskill\Module 3 Core Java>
```

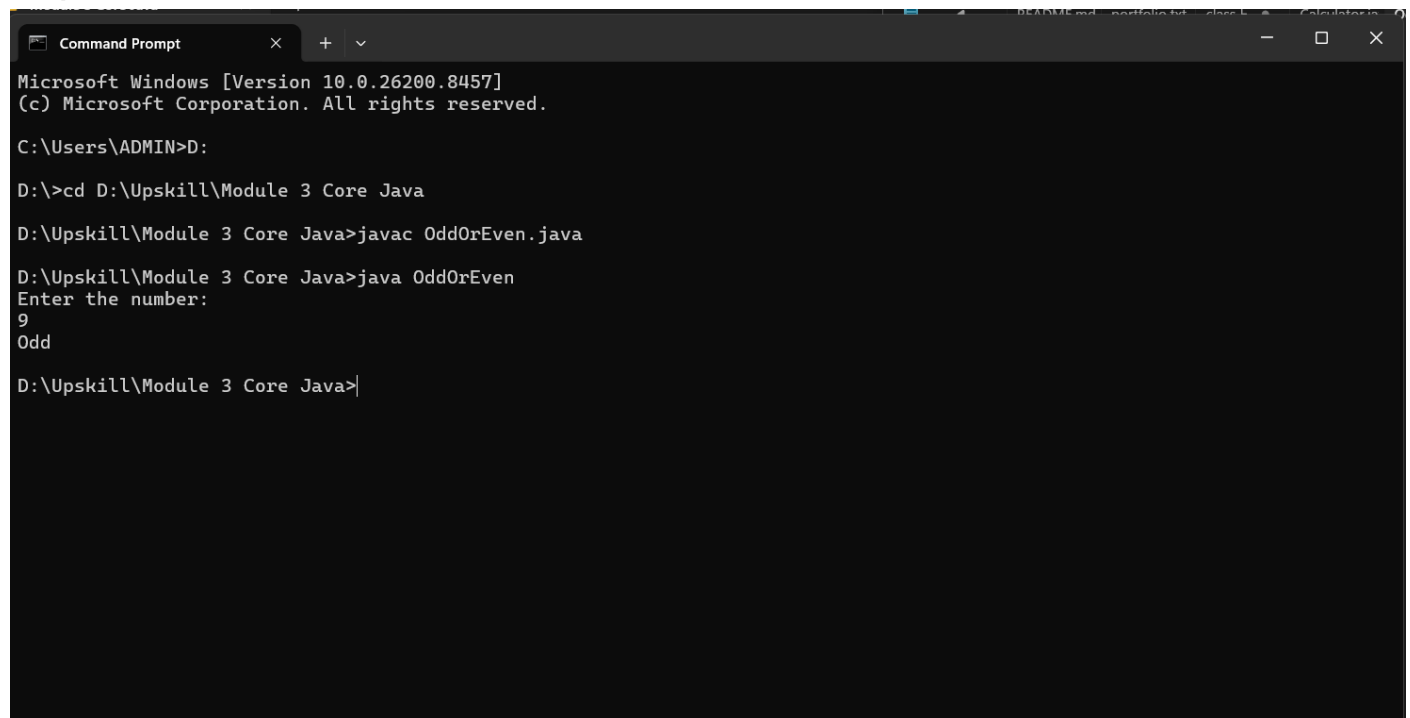
3. Even or Odd Checker

Objective: Utilize conditional statements to make decisions based on input values.

Code:

```
import java.util.Scanner;\n\nclass OddOrEven{\n\n    public static void main(String[] args) {\n\n        Scanner sc=new Scanner(System.in);\n\n        System.out.println("Enter the number: ");\n\n        int n=sc.nextInt();\n\n        if(n%2==0) System.out.println("Even");\n\n        else System.out.println("Odd");\n\n    }\n}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac OddOrEven.java

D:\Upskill\Module 3 Core Java>java OddOrEven
Enter the number:
9
Odd

D:\Upskill\Module 3 Core Java>
```

4. Leap Year Checker

Objective: Apply nested conditional logic to solve real-world problems.

Code:

```
import java.util.*;

public class Leap{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a year: ");

        int y=sc.nextInt();

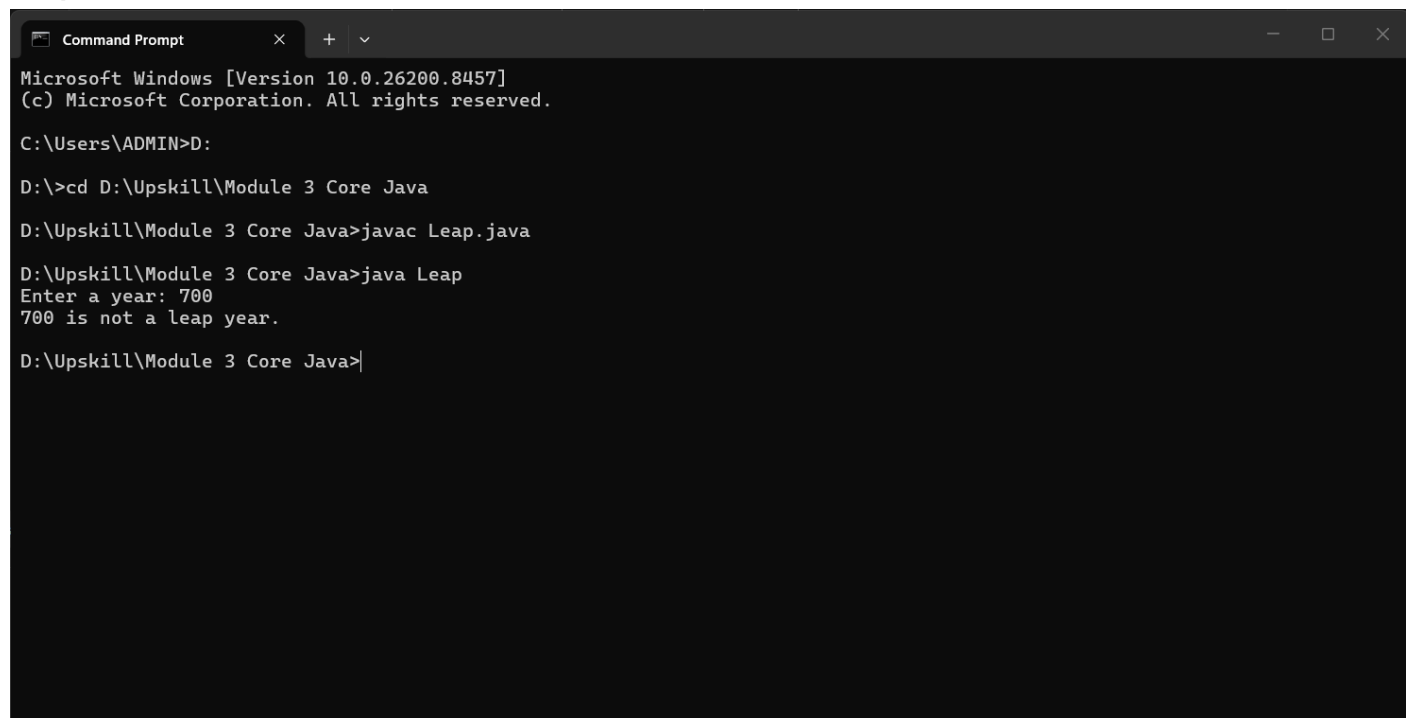
        if((y%4==0) && (y%100!=0) || (y%400==0)) System.out.println(y+" is a leap year.");

        else System.out.println(y+" is not a leap year.");

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Leap.java

D:\Upskill\Module 3 Core Java>java Leap
Enter a year: 700
700 is not a leap year.

D:\Upskill\Module 3 Core Java>|
```

5. Multiplication Table

Objective: Implement loops for repetitive tasks and iteration.

Code:

```
import java.util.*;

public class Table{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a number: ");

        int n=sc.nextInt();

        for(int i=1;i<=10;i++){

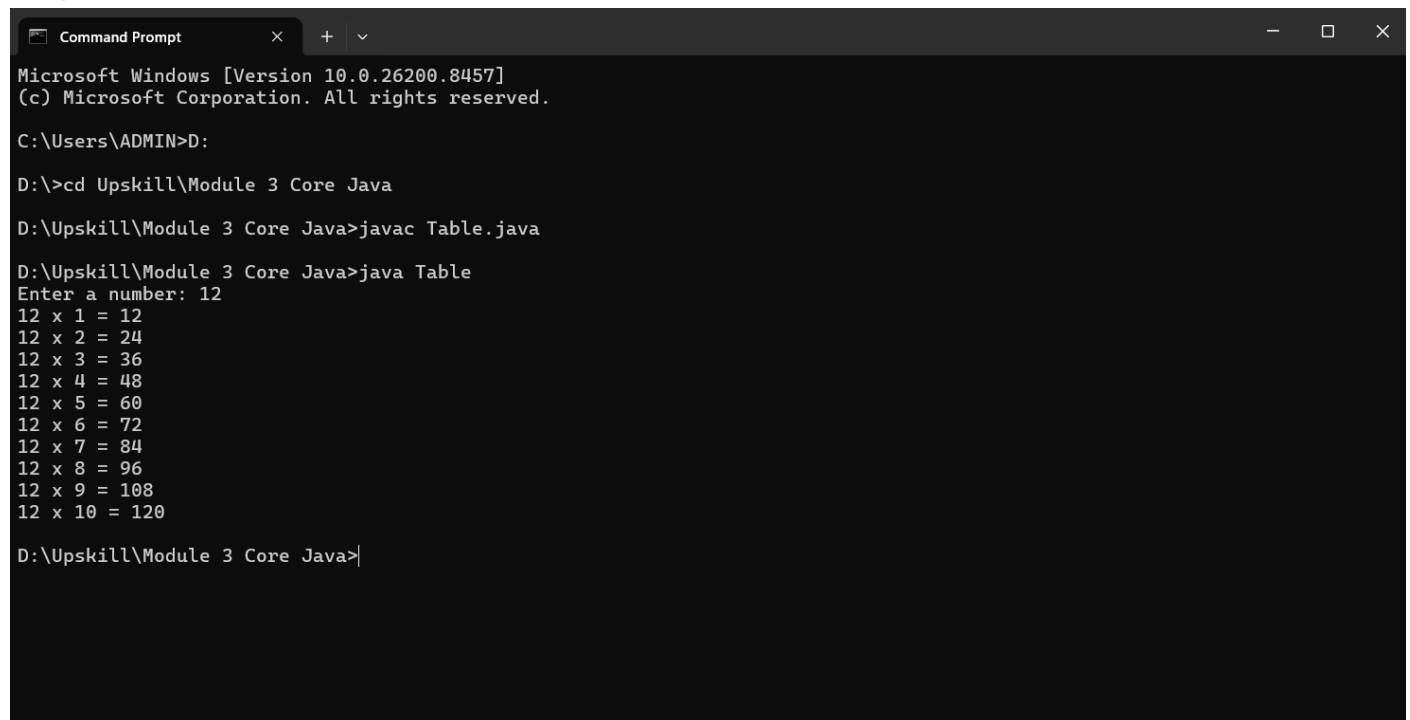
            System.out.println(n+" x "+i+" = "+(n * i));

        }

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Table.java

D:\Upskill\Module 3 Core Java>java Table
Enter a number: 12
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
12 x 4 = 48
12 x 5 = 60
12 x 6 = 72
12 x 7 = 84
12 x 8 = 96
12 x 9 = 108
12 x 10 = 120

D:\Upskill\Module 3 Core Java>
```

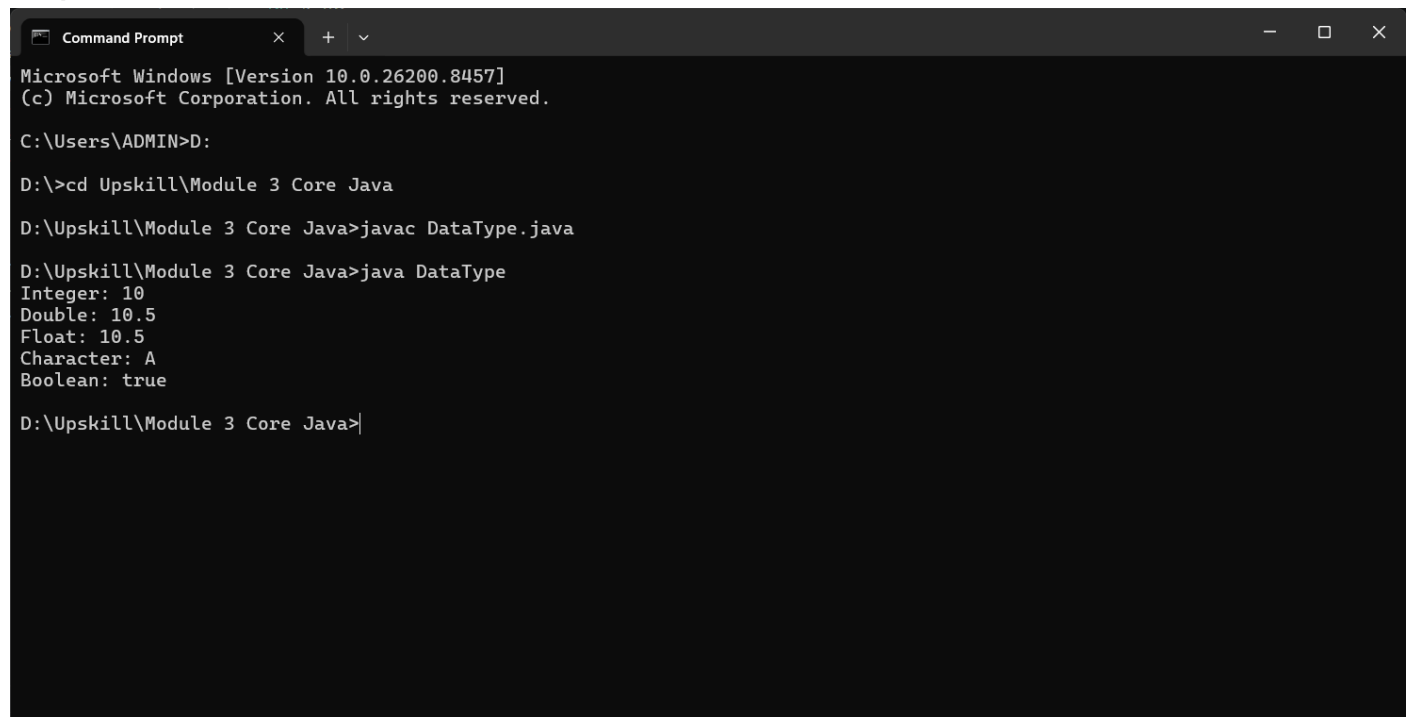
6. Data Type Demonstration

Objective: Understand Java's primitive data types and their usage.

Code:

```
public class DataType {  
  
    public static void main(String[] args){  
  
        int a=10;  
  
        double b=10.5;  
  
        float c=10.5f;  
  
        char d='A';  
  
        boolean e=true;  
  
        System.out.println("Integer: "+a);  
  
        System.out.println("Double: "+b);  
  
        System.out.println("Float: "+c);  
  
        System.out.println("Character: "+d);  
  
        System.out.println("Boolean: "+e);  
  
    }  
  
}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac DataType.java

D:\Upskill\Module 3 Core Java>java DataType
Integer: 10
Double: 10.5
Float: 10.5
Character: A
Boolean: true

D:\Upskill\Module 3 Core Java>
```

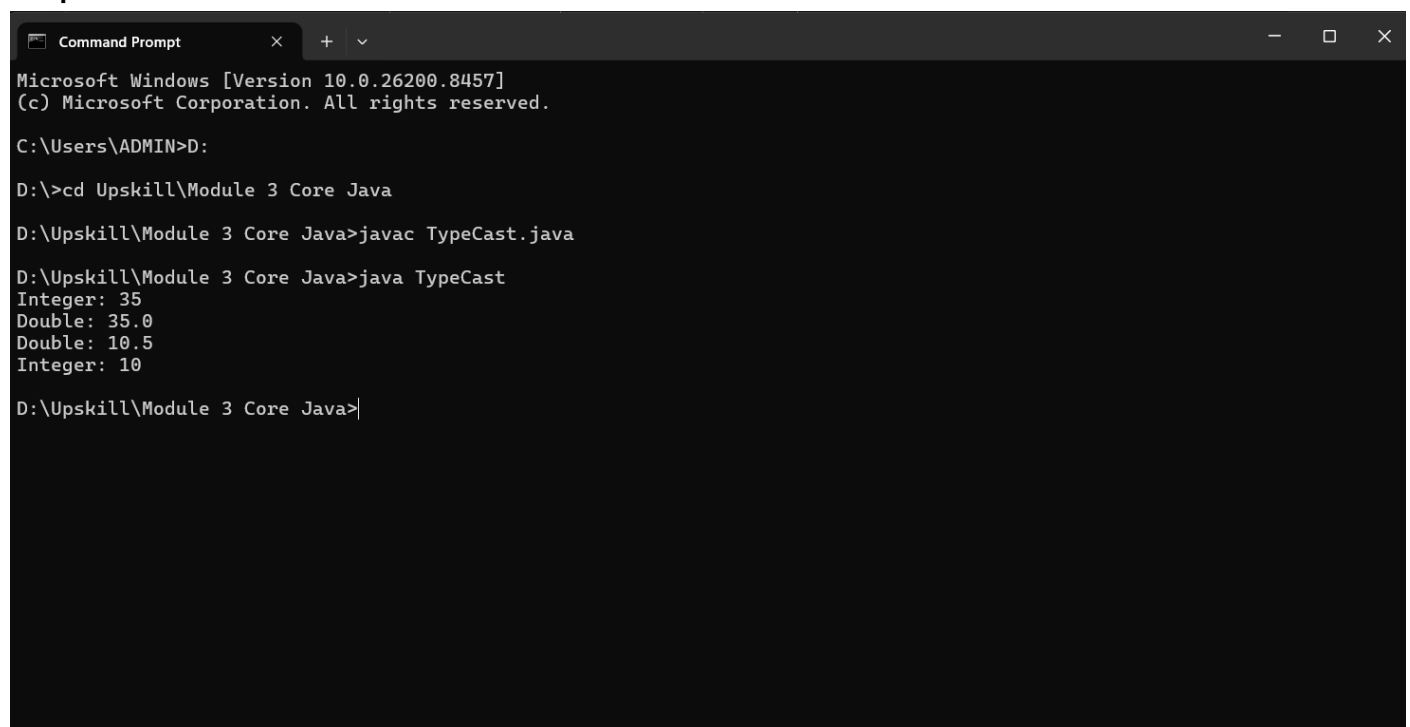

7. Type Casting Example

Objective: Practice converting data between different data types using type casting.

Code:

```
public class TypeCast{  
    public static void main(String[] args){  
        int a=35;  
        double b=a;  
        System.out.println("Integer: "+a);  
        System.out.println("Double: "+b);  
        double c=10.5;  
        int d=(int)c;  
        System.out.println("Double: "+c);  
        System.out.println("Integer: "+d);  
    }  
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>cd Upskill\Module 3 Core Java  
  
D:\Upskill\Module 3 Core Java>javac TypeCast.java  
  
D:\Upskill\Module 3 Core Java>java TypeCast  
Integer: 35  
Double: 35.0  
Double: 10.5  
Integer: 10  
  
D:\Upskill\Module 3 Core Java>
```

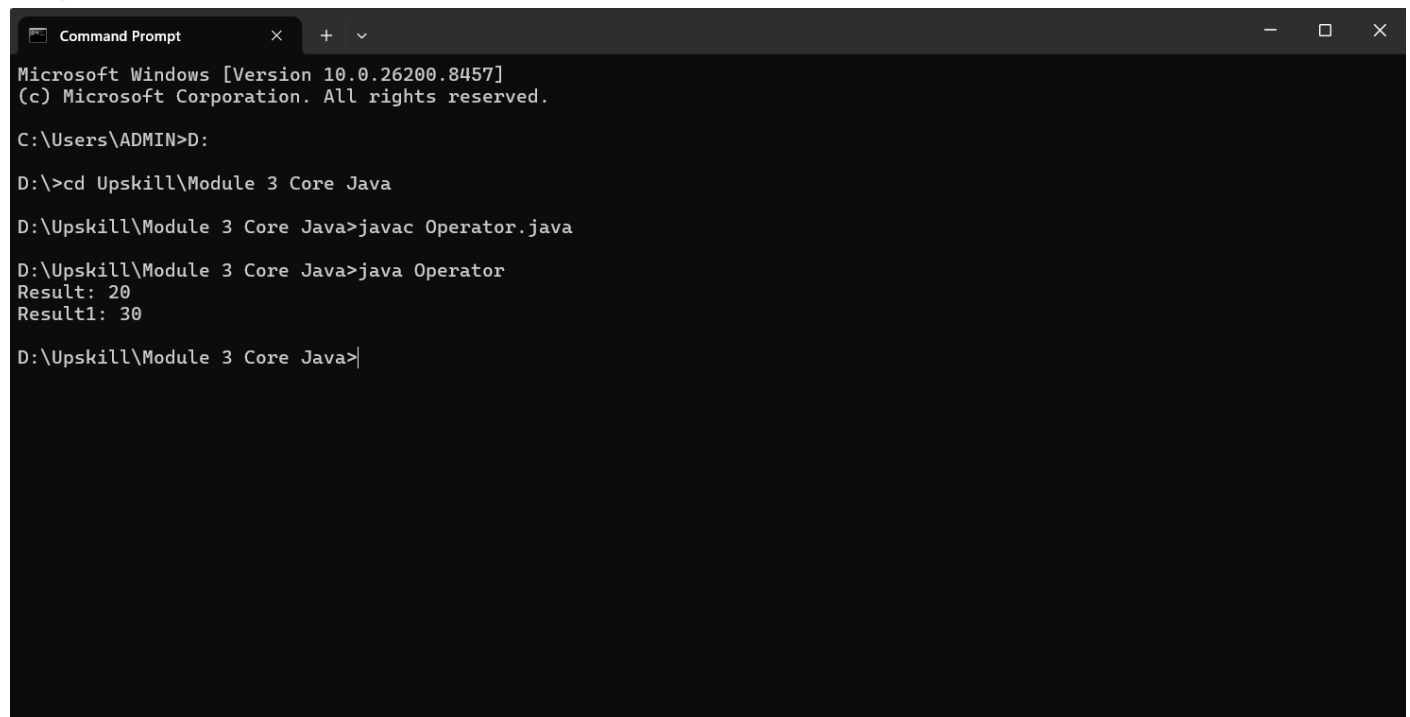
8. Operator Precedence

Objective: Explore how Java evaluates expressions and follows operator precedence rules.

Code:

```
public class Operator{  
  
    public static void main(String[] args){  
  
        int result = 10+5*2;  
  
        System.out.println("Result: " + result);  
  
        int result1=(10+5)*2;  
  
        System.out.println("Result1: "+result1);  
  
    }  
  
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>cd Upskill\Module 3 Core Java  
  
D:\Upskill\Module 3 Core Java>javac Operator.java  
  
D:\Upskill\Module 3 Core Java>java Operator  
Result: 20  
Result1: 30  
  
D:\Upskill\Module 3 Core Java>
```

9. Grade Calculator

Objective: Use conditional statements to classify data and determine grades.

Code:

```
import java.util.*;

public class Grade{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter marks: ");

        int m=sc.nextInt();

        if( m<=100 && m>=90) System.out.println("Grade: A");

        else if(m<=89 && m>=80) System.out.println("Grade: B");

        else if(m<=79 && m>=70) System.out.println("Grade: C");

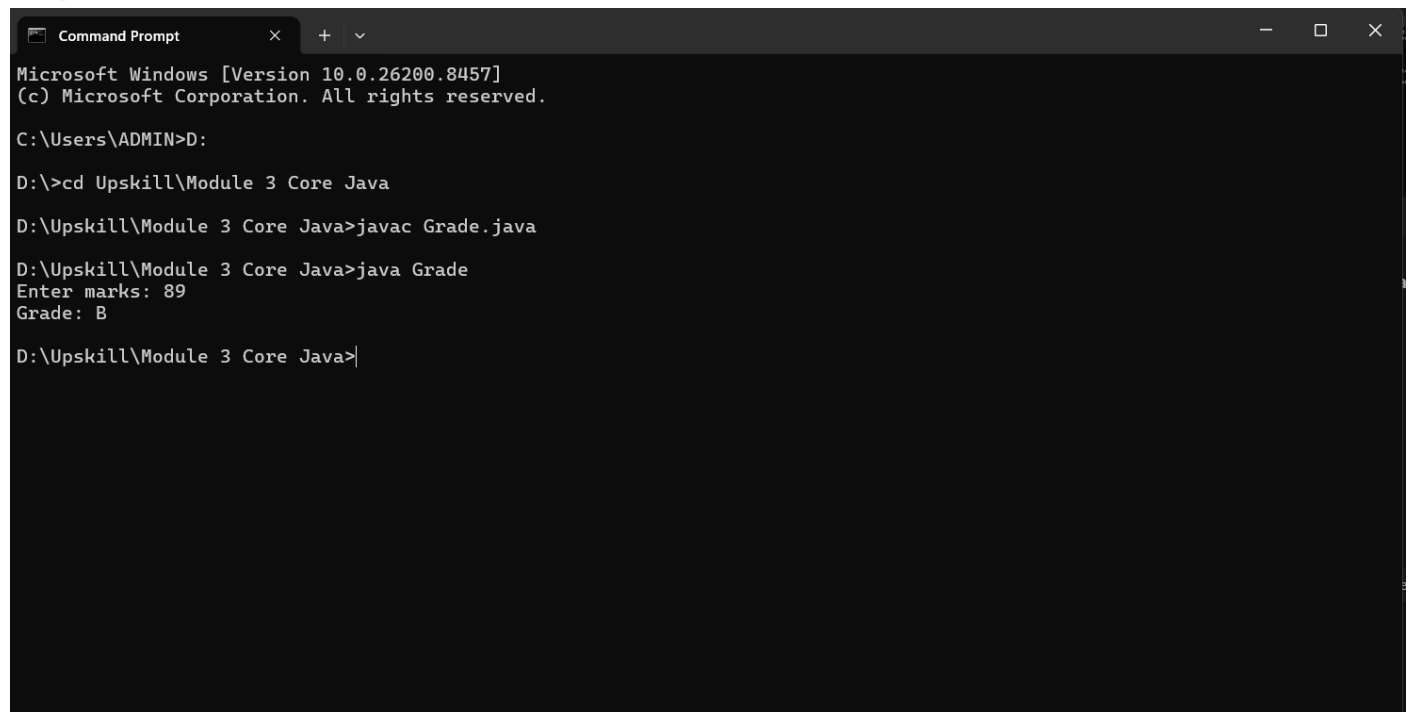
        else if(m<=69 && m>=60) System.out.println("Grade: D");

        else System.out.println("Grade: F");

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Grade.java

D:\Upskill\Module 3 Core Java>java Grade
Enter marks: 89
Grade: B

D:\Upskill\Module 3 Core Java>
```

10. Number Guessing Game

Objective: Implement loops and conditional logic in an interactive program.

Code:

```
import java.util.Scanner;

import java.util.Random;

public class NumberGuess{

    public static void main(String[] args){

        Random rand=new Random();

        Scanner sc=new Scanner(System.in);

        int n=rand.nextInt(100)+1;

        int g=0;

        while(g!=n){

            System.out.print("Guess a number between 1 and 100: ");

            g=sc.nextInt();

            if(g<n) System.out.println("Too low! Try again.");

            else if(g>n) System.out.println("Too high! Try again.");

            else System.out.println("Congratulations! You guessed the number.");

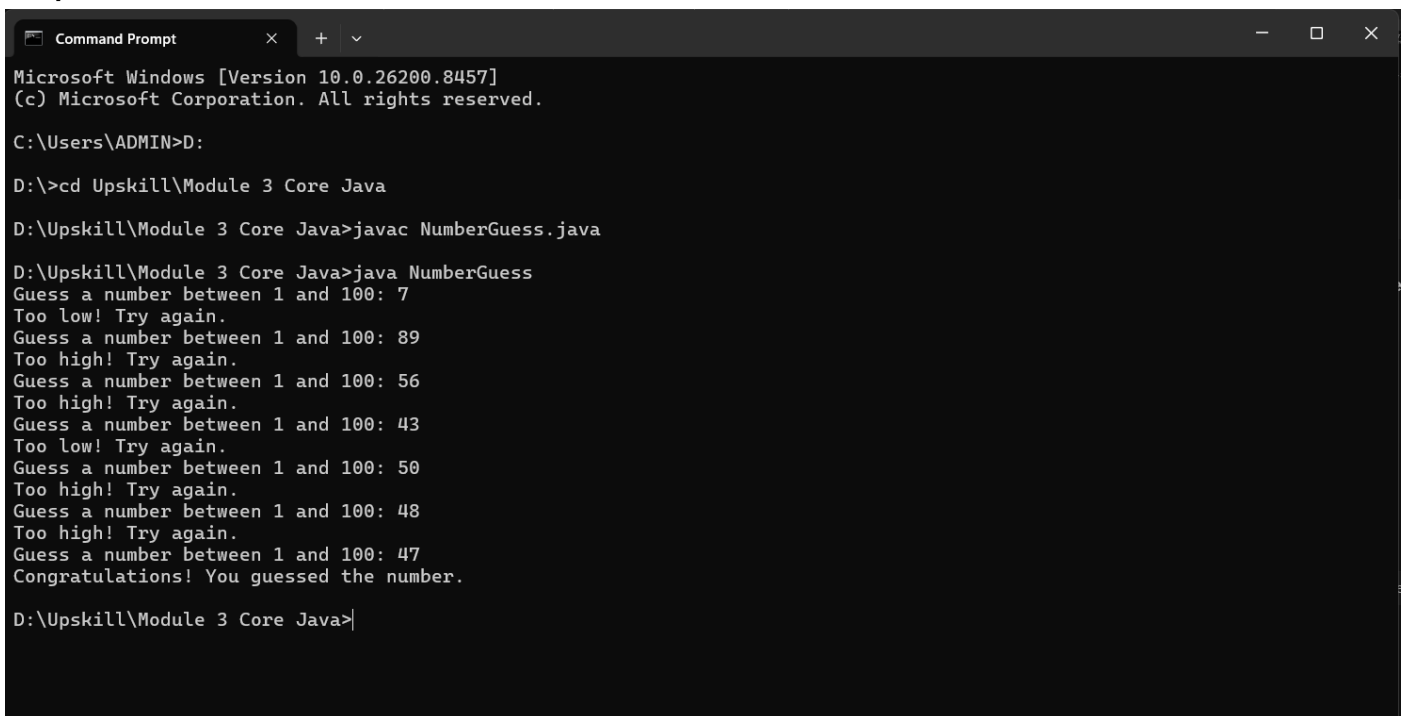
        }

        sc.close();

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac NumberGuess.java

D:\Upskill\Module 3 Core Java>java NumberGuess
Guess a number between 1 and 100: 7
Too low! Try again.
Guess a number between 1 and 100: 89
Too high! Try again.
Guess a number between 1 and 100: 56
Too high! Try again.
Guess a number between 1 and 100: 43
Too low! Try again.
Guess a number between 1 and 100: 50
Too high! Try again.
Guess a number between 1 and 100: 48
Too high! Try again.
Guess a number between 1 and 100: 47
Congratulations! You guessed the number.

D:\Upskill\Module 3 Core Java>
```

11. Factorial Calculator

Objective: Use loops to perform repetitive calculations efficiently.

Code:

```
import java.util.Scanner;

public class Factorial{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a number: ");

        int n=sc.nextInt();

        long fact=1;

        for(int i=1; i<=n; i++){

            fact*=i;

        }

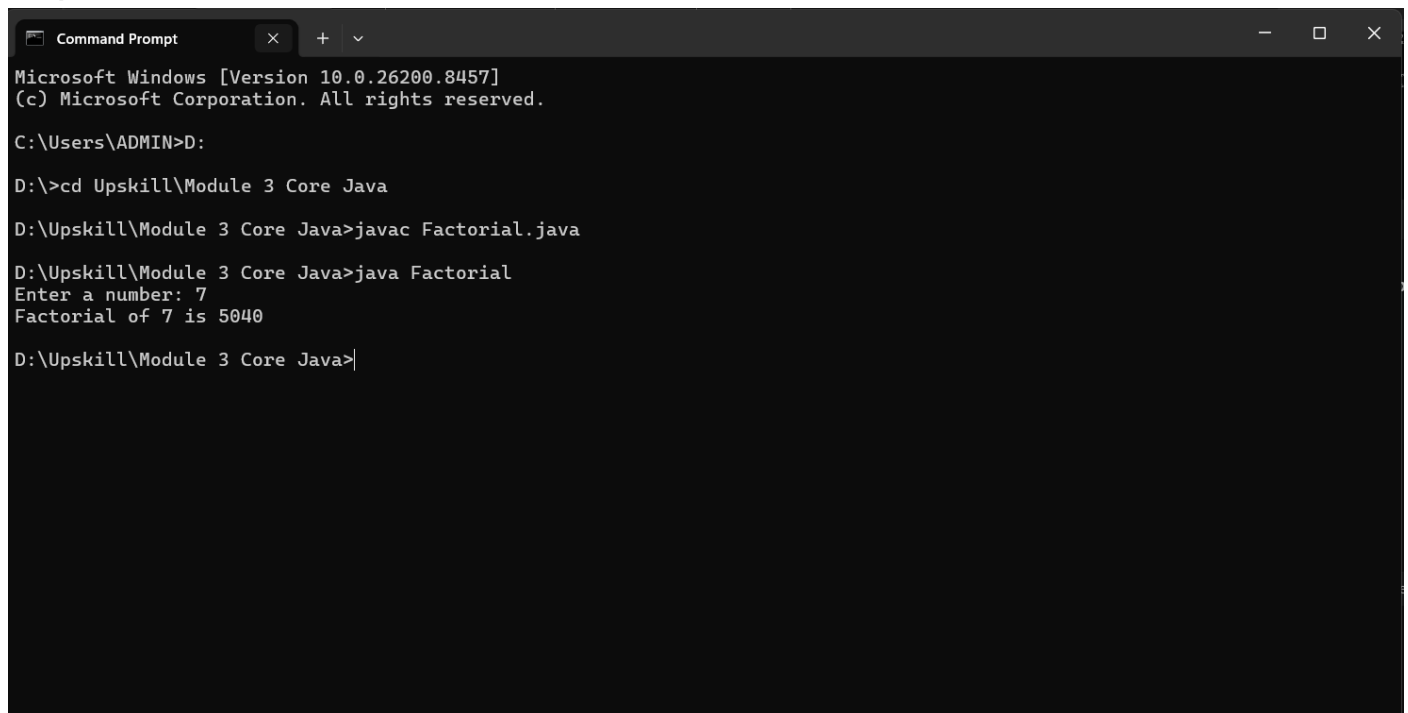
        System.out.println("Factorial of "+n+" is "+fact);

        sc.close();

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Factorial.java

D:\Upskill\Module 3 Core Java>java Factorial
Enter a number: 7
Factorial of 7 is 5040

D:\Upskill\Module 3 Core Java>
```

12. Method Overloading

Objective: Understand method overloading and compile-time polymorphism in Java.

Code:

```
public class Overloading{

    public static void main(String[] args){

        System.out.println("Sum of 2 and 3: "+sum(2,3));

        System.out.println("Sum of 2.5 and 3.5: "+sum(2.5,3.5));

        System.out.println("Sum of 1, 2 and 3: "+sum(1,2,3));

    }

    public static int sum(int a,int b){

        return a+b;

    }

    public static double sum(double a,double b){

        return a+b;

    }

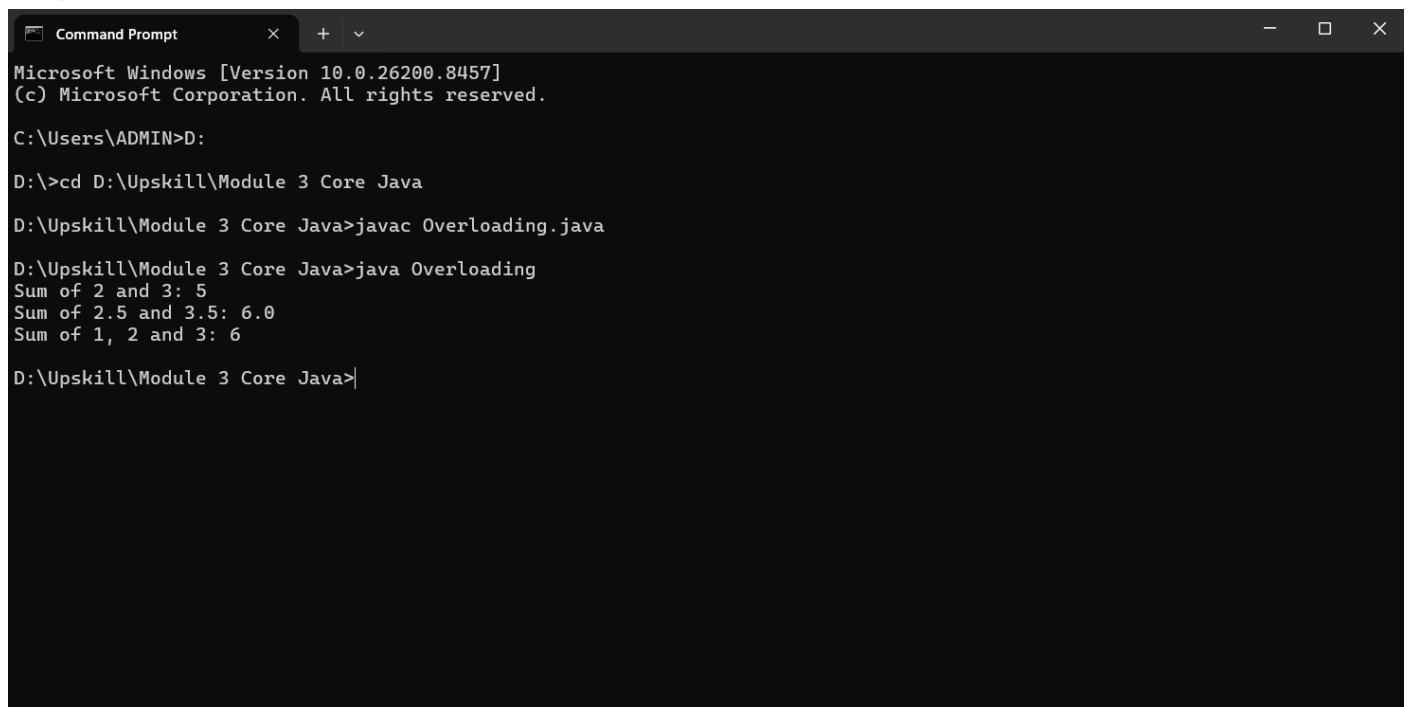
    public static int sum(int a,int b,int c){

        return a+b+c;

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Overloading.java

D:\Upskill\Module 3 Core Java>java Overloading
Sum of 2 and 3: 5
Sum of 2.5 and 3.5: 6.0
Sum of 1, 2 and 3: 6

D:\Upskill\Module 3 Core Java>
```

13. Recursive Fibonacci

Objective: Implement recursion to solve mathematical and sequence-based problems.

Code:

```
import java.util.Scanner;

public class Fibanocci{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter the number: ");

        int n=sc.nextInt();

        int ans=fibonacci(n);

        System.out.println("Fibonacci of "+n+"th term is "+ans);

        sc.close();

    }

    public static int fibonacci(int n){

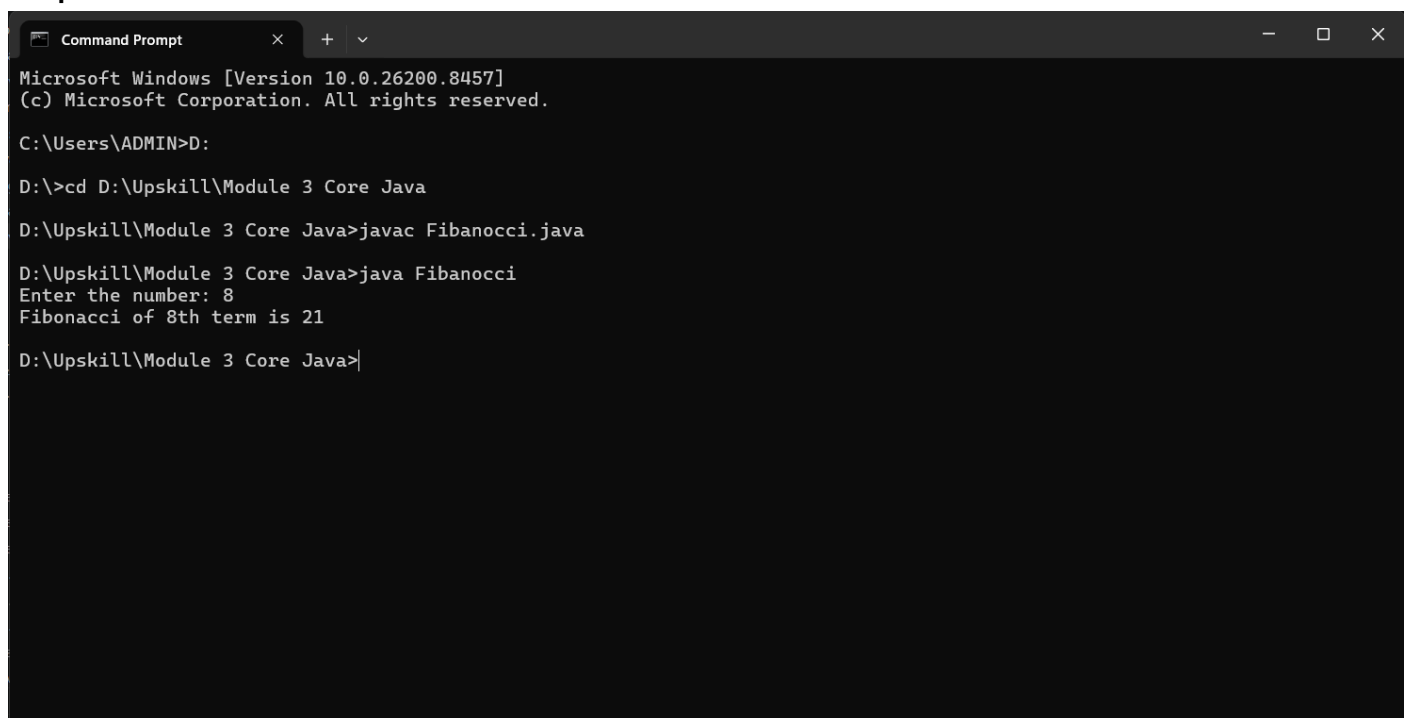
        if(n<=1) return n;

        return fibonacci(n-1)+fibonacci(n-2);

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Fibanocci.java

D:\Upskill\Module 3 Core Java>java Fibanocci
Enter the number: 8
Fibonacci of 8th term is 21

D:\Upskill\Module 3 Core Java>
```

14. Array Sum and Average

Objective: Work with arrays and perform aggregate calculations on stored data.

Code:

```
import java.util.Scanner;

public class Sum{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter number: ");

        int n=sc.nextInt();

        int[] arr=new int[n];

        int sum=0;

        System.out.println("Enter Array Elements: ");

        for(int i=0; i<n; i++){

            arr[i]=sc.nextInt();

            sum+=arr[i];

        }

        System.out.println("Sum of Array Elements: "+sum);

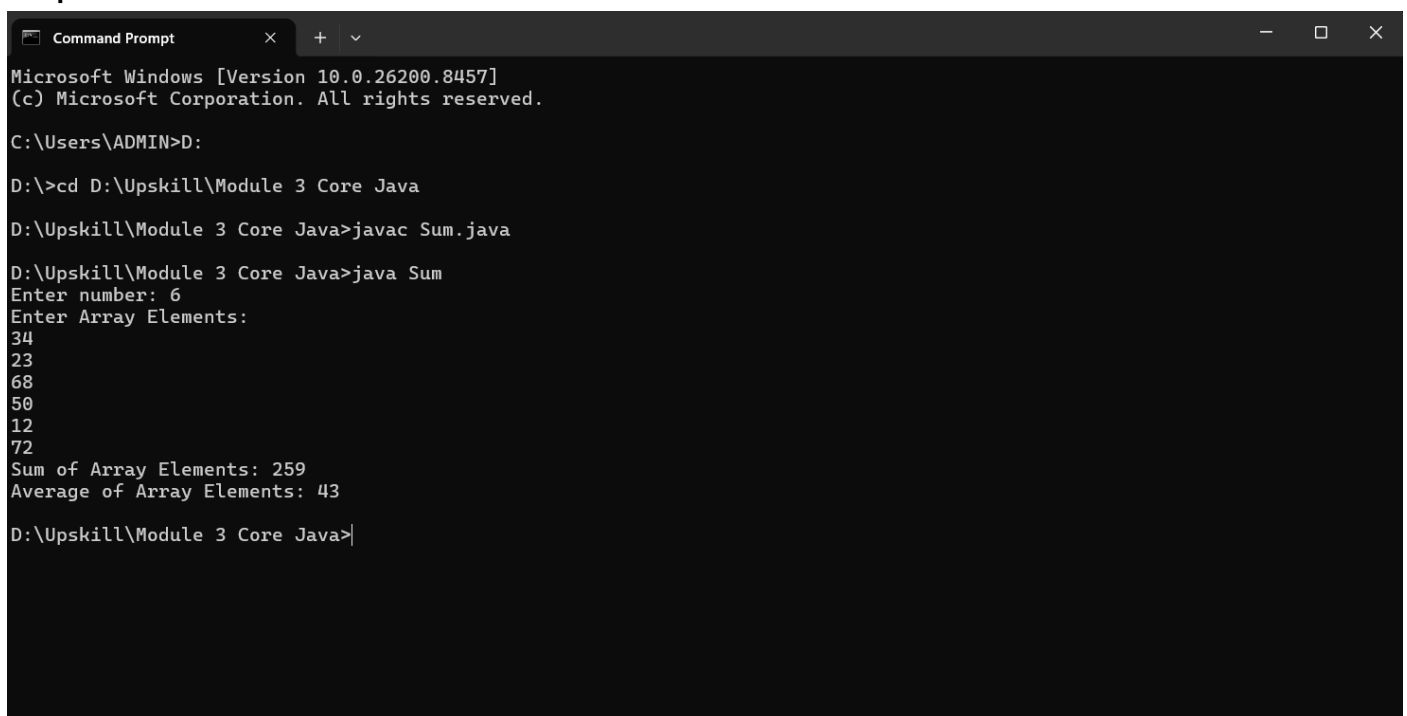
        System.out.println("Average of Array Elements: "+(sum/n));

        sc.close();

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Sum.java

D:\Upskill\Module 3 Core Java>java Sum
Enter number: 6
Enter Array Elements:
34
23
68
50
12
72
Sum of Array Elements: 259
Average of Array Elements: 43

D:\Upskill\Module 3 Core Java>
```


15. String Reversal

Objective: Manipulate strings and understand string-processing techniques.

Code:

```
import java.util.Scanner;

public class Reverse{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a String:");

        String s=sc.nextLine();

        String rev="";

        for(int i=s.length()-1;i>=0;i--){

            rev+=s.charAt(i);

        }

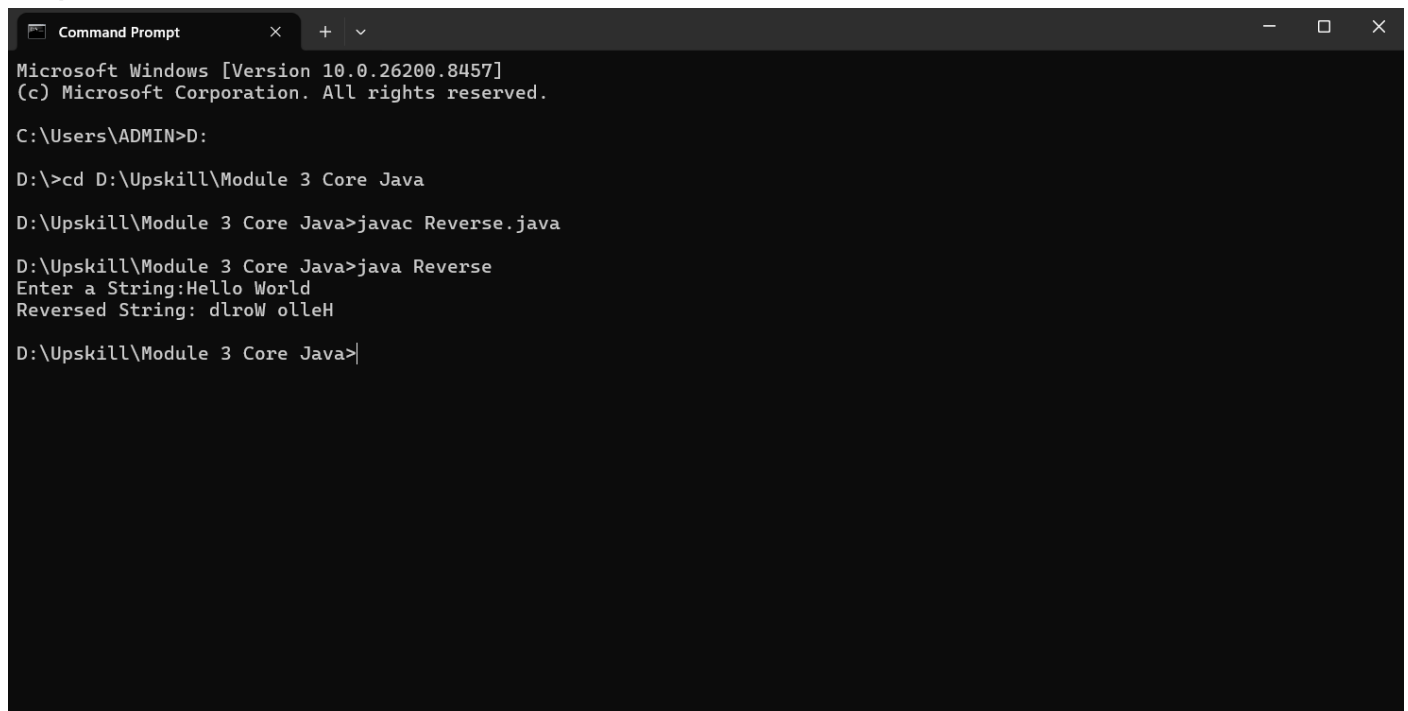
        System.out.println("Reversed String: "+rev);

        sc.close();

    }

}
```

Output:

A screenshot of a Windows Command Prompt window. The title bar says "Command Prompt". The window content shows the following text:
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.
C:\Users\ADMIN>D:
D:\>cd D:\Upskill\Module 3 Core Java
D:\Upskill\Module 3 Core Java>javac Reverse.java
D:\Upskill\Module 3 Core Java>java Reverse
Enter a String:Hello World
Reversed String: dlroW olleH
D:\Upskill\Module 3 Core Java>|

16. Palindrome Checker

Objective: Combine string manipulation with conditional logic to validate patterns.

Code:

```
import java.util.*;

public class Palindrome{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter a String: ");

        String s=sc.nextLine();

        String str=s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();

        if(reverse(str)) System.out.println(s+" is a palindrome.");

        else System.out.println(s+" is not a palindrome.");

        sc.close();

    }

    public static boolean reverse(String str){

        int l=0;

        int r=str.length()-1;

        while(l<r){

            if(str.charAt(l)!=str.charAt(r)){

                return false;

            }

            l++;

            r--;

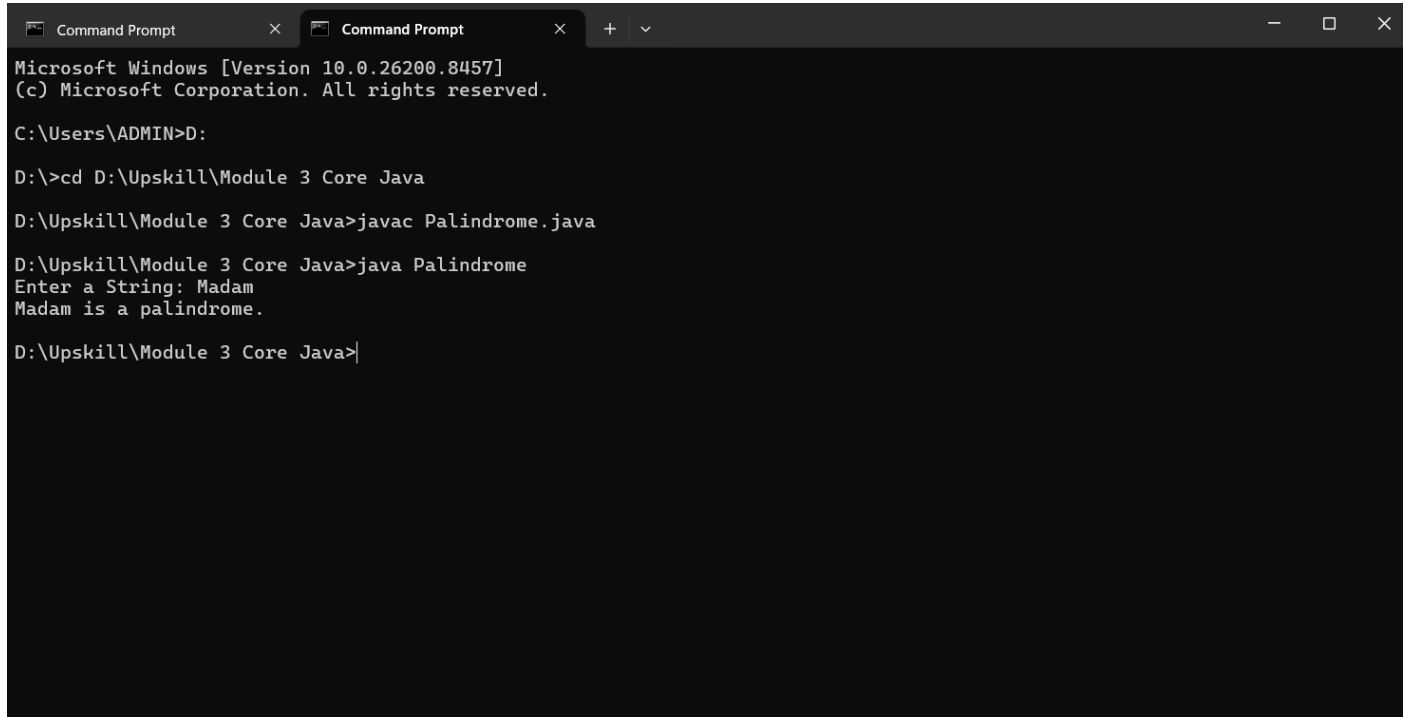
        }

        return true;

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Module 3 Core Java

D:\Upskill\Module 3 Core Java>javac Palindrome.java

D:\Upskill\Module 3 Core Java>java Palindrome
Enter a String: Madam
Madam is a palindrome.

D:\Upskill\Module 3 Core Java>|
```

17. Class and Object Creation

Objective: Understand object-oriented programming concepts through classes and objects.

Code:

```
public class Car{

    String make;

    String model;

    int year;

    public Car(String make,String model,int year){

        this.make=make;

        this.model=model;

        this.year=year;

    }

    public void displayDetails(){

        System.out.println("Make: "+make);

        System.out.println("Model: "+model);

        System.out.println("Year: "+year);

    }

    public static void main(String[] args){

        Car car1=new Car("Toyota","Camry",2020);

        Car car2=new Car("Honda","Civic",2019);

        car1.displayDetails();

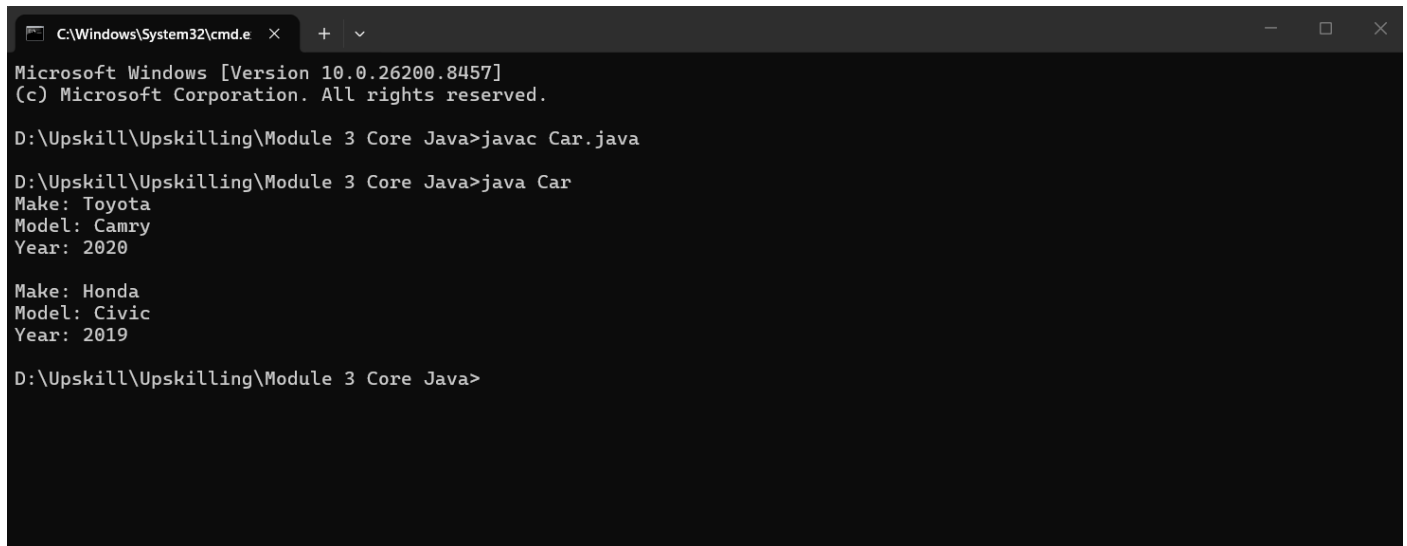
        System.out.println();

        car2.displayDetails();

    }

}
```

Output:



```
C:\Windows\System32\cmd.e  ×  +  ▾  
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
D:\Upskill\Upskilling\Module 3 Core Java>javac Car.java  
  
D:\Upskill\Upskilling\Module 3 Core Java>java Car  
Make: Toyota  
Model: Camry  
Year: 2020  
  
Make: Honda  
Model: Civic  
Year: 2019  
  
D:\Upskill\Upskilling\Module 3 Core Java>
```

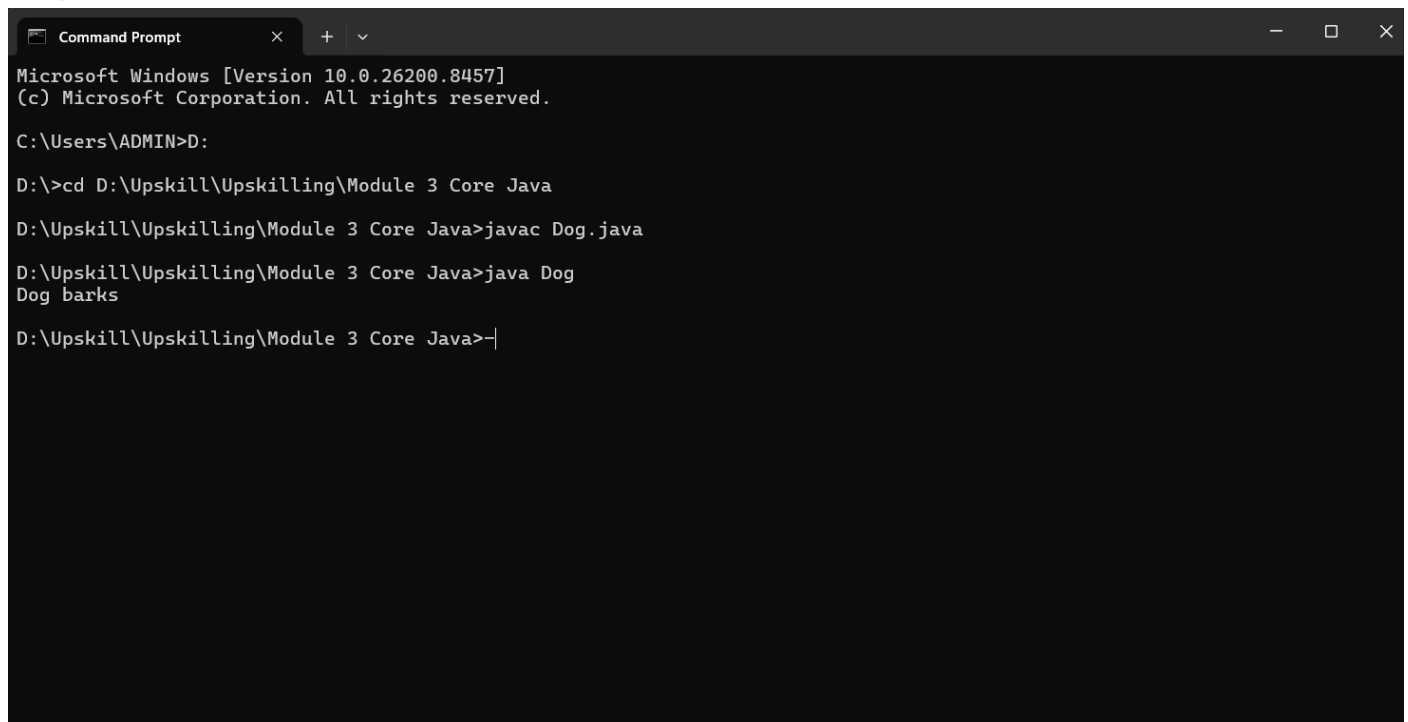
18. Inheritance Example

Objective: Implement inheritance and method overriding to promote code reuse.

Code:

```
class Animal{  
    public void makeSound(){  
        System.out.println("Animal makes a sound");  
    }  
}  
  
public class Dog extends Animal{  
    public void makeSound(){  
        System.out.println("Dog barks");  
    }  
  
    public static void main(String[] args){  
        Animal dog=new Dog();  
        dog.makeSound();  
    }  
}
```

Output:



```
Command Prompt  
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>cd D:\Upskill\Upskilling\Module 3 Core Java  
  
D:\Upskill\Upskilling\Module 3 Core Java>javac Dog.java  
  
D:\Upskill\Upskilling\Module 3 Core Java>java Dog  
Dog barks  
  
D:\Upskill\Upskilling\Module 3 Core Java>|
```

19. Interface Implementation

Objective: Use interfaces to achieve abstraction and multiple-behavior design.

Code:

```
interface Playable{

    void play();

}

class Guitar implements Playable{

    public void play(){

        System.out.println("Guitar is playing");

    }

}

class Piano implements Playable{

    public void play(){

        System.out.println("Piano is playing");

    }

}

public class Music{

    public static void main(String[] args){

        Playable guitar=new Guitar();

        Playable piano=new Piano();

        guitar.play();

        piano.play();

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Music.java

D:\Upskill\Upskilling\Module 3 Core Java>java Music
Guitar is playing
Piano is playing

D:\Upskill\Upskilling\Module 3 Core Java>|
```

20. Try-Catch Example

Objective: Handle exceptions gracefully and improve program reliability.

Code:

```
import java.util.Scanner;

public class Try{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        try{

            System.out.print("Enter a number:");

            int n=sc.nextInt();

            System.out.print("Enter a number:");

            int m=sc.nextInt();

            System.out.println("Result: "+(n/m));

        }

        catch(ArithmeticException e){

            System.out.println("Error: Cannot divide by Zero. Please enter a non-zero number.");

        }

        finally{

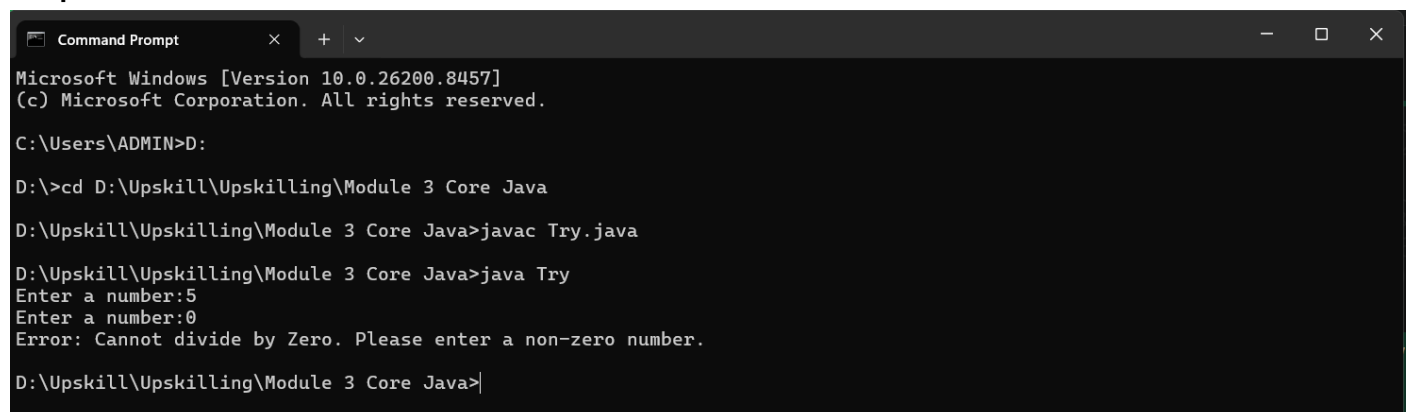
            sc.close();

        }

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java
D:\Upskill\Upskilling\Module 3 Core Java>javac Try.java
D:\Upskill\Upskilling\Module 3 Core Java>java Try
Enter a number:5
Enter a number:0
Error: Cannot divide by Zero. Please enter a non-zero number.

D:\Upskill\Upskilling\Module 3 Core Java>
```


21. Custom Exception

Objective: Create and use custom exceptions for application-specific error handling.

Code:

```
import java.util.Scanner;

class InvalidAgeException extends Exception{

    public InvalidAgeException(String msg){

        super(msg);

    }

}

public class CustomException{

    public static void checkAge(int age) throws InvalidAgeException{

        if(age<18)

            throw new InvalidAgeException("Invalid age: "+age+". Age must be 18 or above. ");

        else

            System.out.println("Valid age: "+age);

    }

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        try{

            System.out.print("Enter your age: ");

            int age=sc.nextInt();

            checkAge(age);

        }catch(InvalidAgeException e){

            System.out.println("Error: "+e.getMessage());

        }finally{

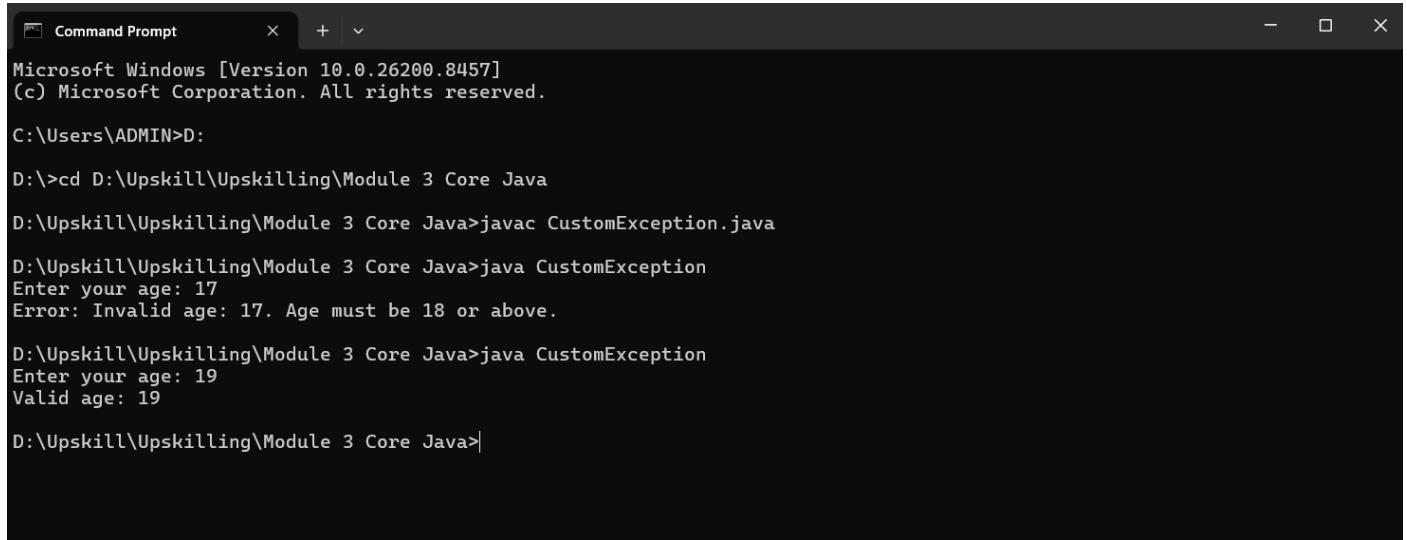
            sc.close();

        }

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac CustomException.java

D:\Upskill\Upskilling\Module 3 Core Java>java CustomException
Enter your age: 17
Error: Invalid age: 17. Age must be 18 or above.

D:\Upskill\Upskilling\Module 3 Core Java>java CustomException
Enter your age: 19
Valid age: 19

D:\Upskill\Upskilling\Module 3 Core Java>|
```

22. File Writing

Objective: Learn how to write data to files using Java I/O operations.

Code:

```
import java.io.*;

import java.util.*;

public class FileWrite{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        System.out.println("Enter a text to write to the file: ");

        String text=sc.nextLine();

        try(FileWriter writer=new FileWriter("output.txt")){

            writer.write(text);

            System.out.println("Text written to file successfully.");

        }

        catch(IOException e){

            System.out.println("An error occurred while writing to the file: "+e.getMessage());

        }

        finally{

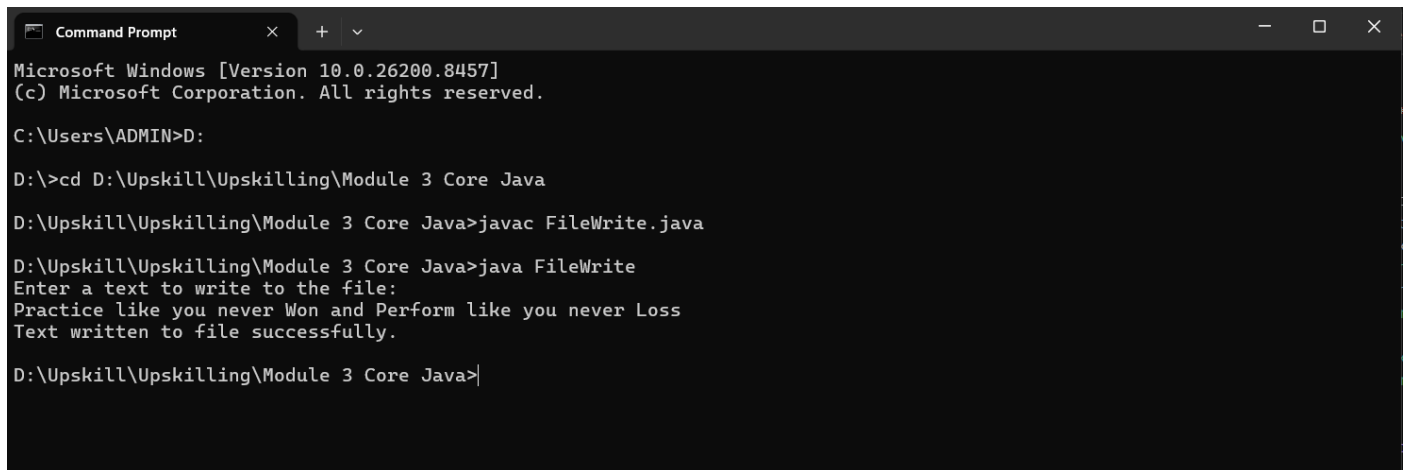
            sc.close();

        }

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac FileWrite.java

D:\Upskill\Upskilling\Module 3 Core Java>java FileWrite
Enter a text to write to the file:
Practice like you never Won and Perform like you never Loss
Text written to file successfully.

D:\Upskill\Upskilling\Module 3 Core Java>
```

23. File Reading

Objective: Learn how to read and process data from files.

Code:

```
import java.io.*;

public class FileRead{

    public static void main(String[] args){

        try(BufferedReader reader=new BufferedReader(new FileReader("output.txt"))){

            String line;

            System.out.println("Contents of the file:");

            while((line=reader.readLine())!=null){

                System.out.println(line);

            }

        }

        catch(IOException e){

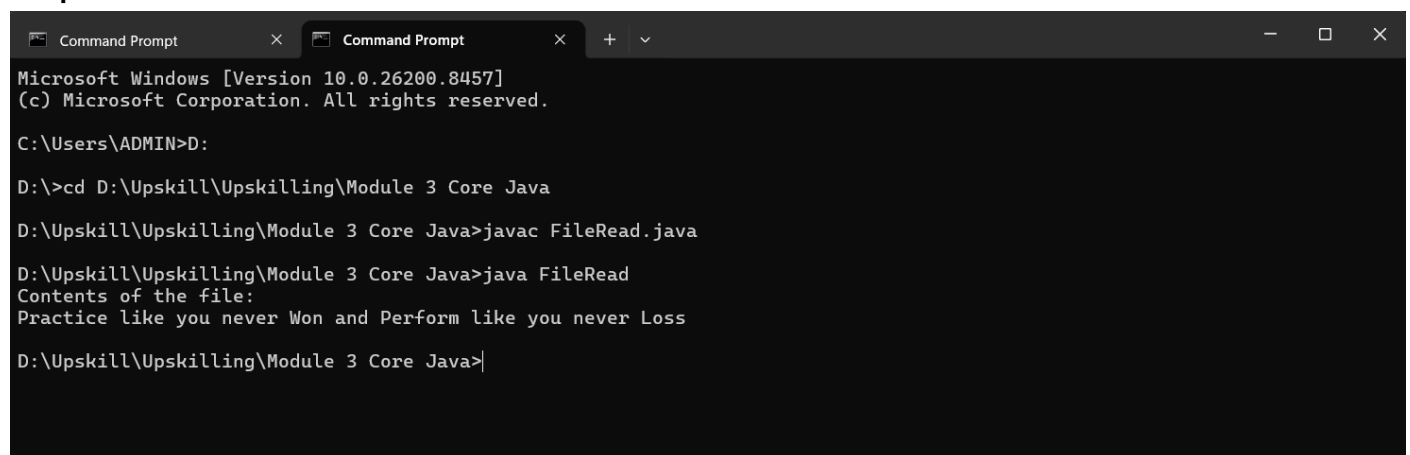
            System.out.println("An error occurred while reading the file: "+e.getMessage());

        }

    }

}
```

Output:

A screenshot of a Windows Command Prompt window. The title bar shows two tabs, both labeled 'Command Prompt'. The window content displays the following text: 'Microsoft Windows [Version 10.0.26200.8457] (c) Microsoft Corporation. All rights reserved. C:\Users\ADMIN>D: D:\>cd D:\Upskill\Upskilling\Module 3 Core Java D:\Upskill\Upskilling\Module 3 Core Java>javac FileRead.java D:\Upskill\Upskilling\Module 3 Core Java>java FileRead Contents of the file: Practice like you never Won and Perform like you never Loss D:\Upskill\Upskilling\Module 3 Core Java>'. The output of the Java program is visible as 'Contents of the file:' followed by a new line and then 'Practice like you never Won and Perform like you never Loss'.

24. ArrayList Example

Objective: Use dynamic arrays to store and manage collections of data.

Code:

```
import java.util.*;

public class Array{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        List<String> students=new ArrayList<>();

        while(true){

            System.out.print("Enter student name(or 'exit' to stop): ");

            String name=sc.nextLine();

            if(name.equalsIgnoreCase("exit")) break;

            students.add(name);

        }

        System.out.println("Student names entered:");

        for(String student : students){

            System.out.println(student);

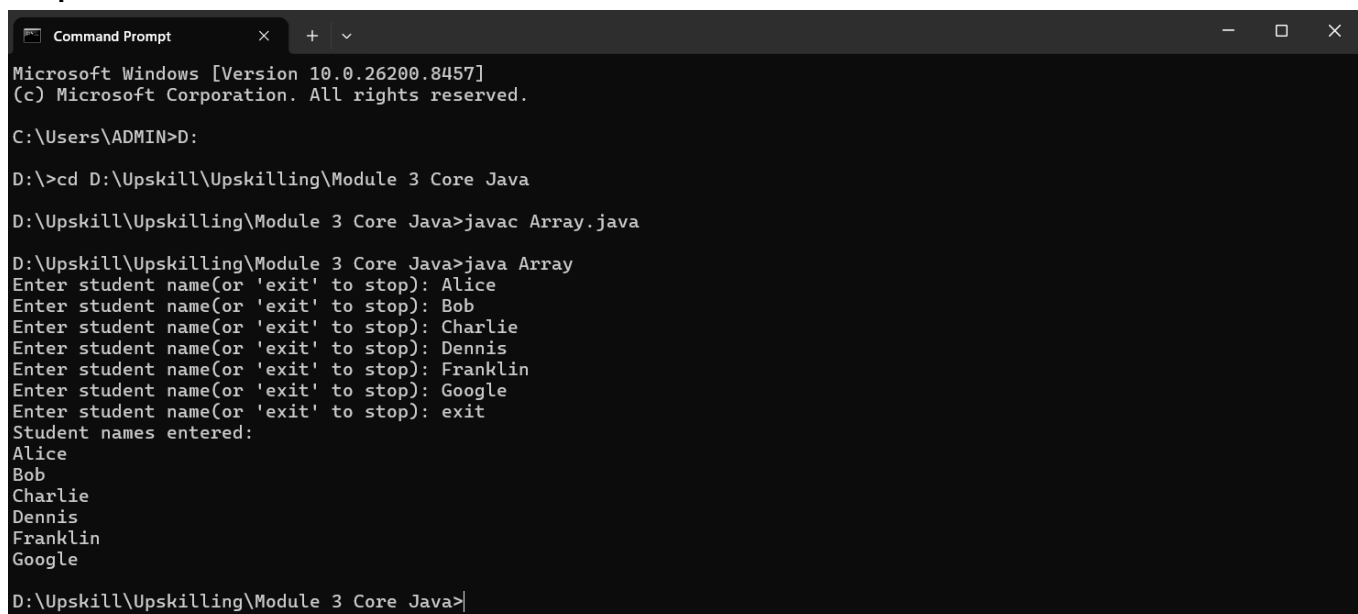
        }

        sc.close();

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Array.java

D:\Upskill\Upskilling\Module 3 Core Java>java Array
Enter student name(or 'exit' to stop): Alice
Enter student name(or 'exit' to stop): Bob
Enter student name(or 'exit' to stop): Charlie
Enter student name(or 'exit' to stop): Dennis
Enter student name(or 'exit' to stop): Franklin
Enter student name(or 'exit' to stop): Google
Enter student name(or 'exit' to stop): exit
Student names entered:
Alice
Bob
Charlie
Dennis
Franklin
Google

D:\Upskill\Upskilling\Module 3 Core Java>
```

25. HashMap Example

Objective: Work with key-value pairs for efficient data storage and retrieval.

Code:

```
import java.util.*;

public class HashMapExample{

    public static void main(String[] args){

        Scanner sc=new Scanner(System.in);

        Map<Integer,String> students=new HashMap<>();

        while(true){

            System.out.print("Enter student ID (or -1 to stop): ");

            int id=sc.nextInt();

            sc.nextLine(); // Consume newline

            if(id==-1) break;

            System.out.print("Enter student name: ");

            String name=sc.nextLine();

            students.put(id,name);

        }

        System.out.println("Enter Id to search for student name: ");

        int sId=sc.nextInt();

        String sName=students.get(sId);

        System.out.println((sName!=null?"Student Name :"+sName: "Student not found for the given Id");

        sc.close();

    }

}
```

Output:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac HashMapExample.java

D:\Upskill\Upskilling\Module 3 Core Java>java HashMapExample
Enter student ID (or -1 to stop): 1
Enter student name: Sai
Enter student ID (or -1 to stop): 2
Enter student name: Madhu
Enter student ID (or -1 to stop): 3
Enter student name: Hari
Enter student ID (or -1 to stop): -1
Enter Id to search for student name:
2
Student Name :Madhu

D:\Upskill\Upskilling\Module 3 Core Java>
```

26. Thread Creation

Objective: Implement multithreading to execute tasks concurrently.

Code:

```
class MyThread extends Thread{

    private String msg;

    public MyThread(String msg){

        this.msg=msg;

    }

    public void run(){

        for(int i=0;i<3;i++){

            System.out.println(Thread.currentThread().getName() + ": " + msg);

            try{

                Thread.sleep(100); // Sleep for 2 seconds

            } catch (InterruptedException e){

                System.out.println("Thread interrupted: " + e.getMessage());

            }

        }

    }

}

public class MultiThreading{

    public static void main(String[] args){

        Thread t1 = new MyThread("Hello from t1");

        Thread t2 = new MyThread("Hello from t2");

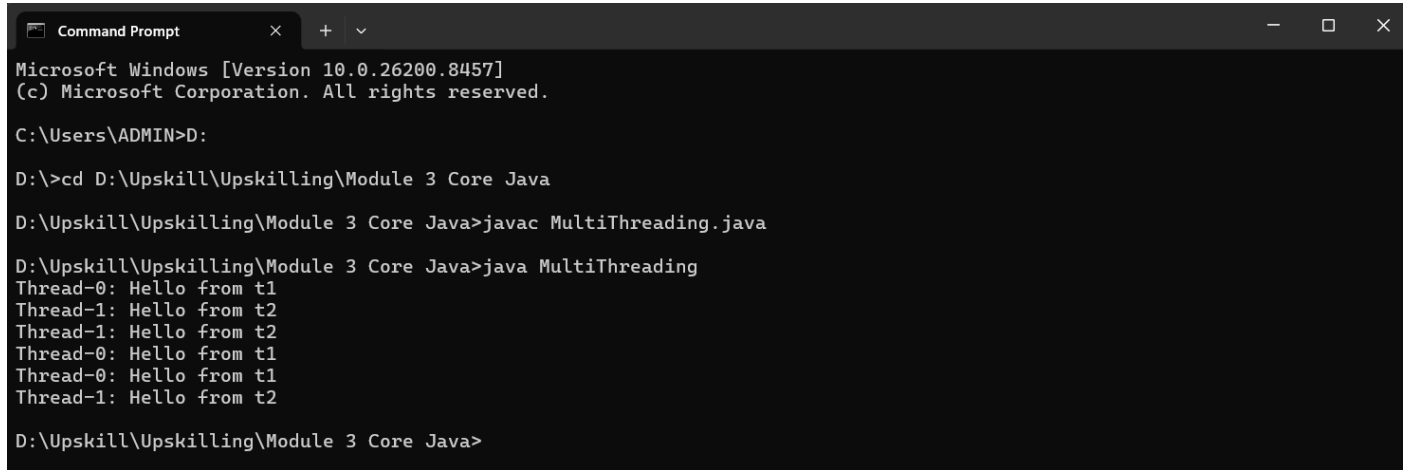
        t1.start();

        t2.start();

    }

}
```


Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac MultiThreading.java

D:\Upskill\Upskilling\Module 3 Core Java>java MultiThreading
Thread-0: Hello from t1
Thread-1: Hello from t2
Thread-1: Hello from t2
Thread-0: Hello from t1
Thread-0: Hello from t1
Thread-1: Hello from t2

D:\Upskill\Upskilling\Module 3 Core Java>
```

27. Lambda Expression

Objective: Use functional programming features introduced in modern Java.

Code:

```
import java.util.*;

public class Fruits{

    public static void main(String[] args){

        List<String>fruits=new ArrayList<>();

        fruits.add("Strewberry");

        fruits.add("Mango");

        fruits.add("Banana");

        fruits.add("Apple");

        fruits.add("Grapes");

        System.out.println("Original List: "+fruits);

        Collections.sort(fruits,

            (s1,s2)->s1.compareTo(s2)

        );

        System.out.println("Sorted List: "+fruits);

        Collections.sort(fruits,

            (s1,s2)->Integer.compare(s1.length(), s2.length())

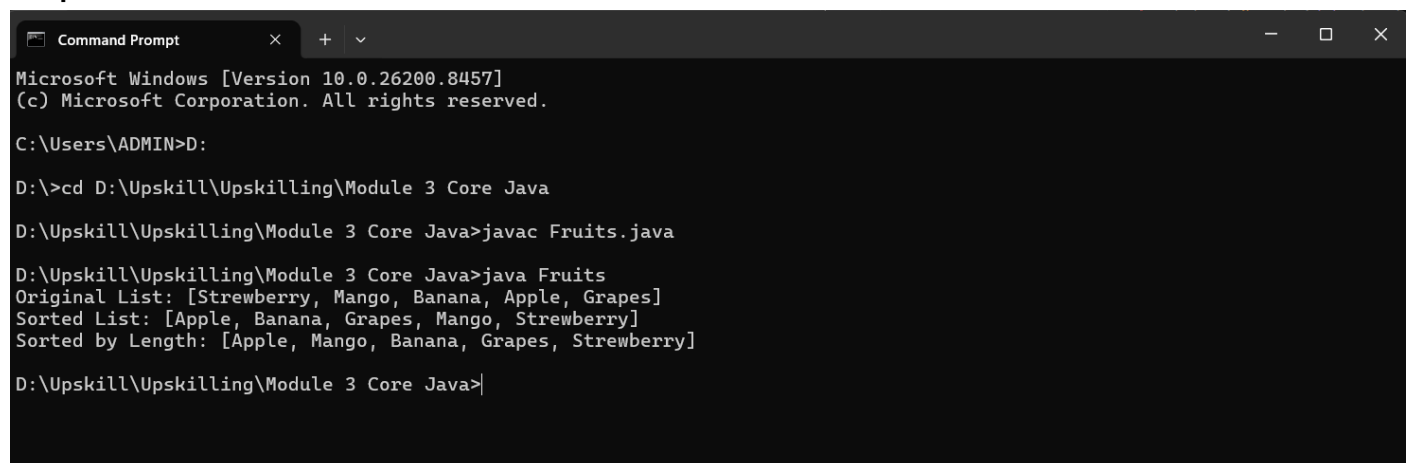
        );

        System.out.println("Sorted by Length: "+fruits);

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Fruits.java

D:\Upskill\Upskilling\Module 3 Core Java>java Fruits
Original List: [Strewberry, Mango, Banana, Apple, Grapes]
Sorted List: [Apple, Banana, Grapes, Mango, Strewberry]
Sorted by Length: [Apple, Mango, Banana, Grapes, Strewberry]

D:\Upskill\Upskilling\Module 3 Core Java>
```

28. Stream API

Objective: Process and manipulate collections efficiently using streams.

Code:

```
import java.util.stream.Collectors;

import java.util.*;

public class Stream{

    public static void main(String[] args){

        List<Integer> numbers=Arrays.asList(1,2,3,4,5,6,7,8,9,10);

        System.out.println("Original List: "+numbers);

        List<Integer> evenNumbers=numbers.stream()

        .filter(n->n%2==0).collect(Collectors.toList());

        System.out.println("Even Numbers: "+evenNumbers);

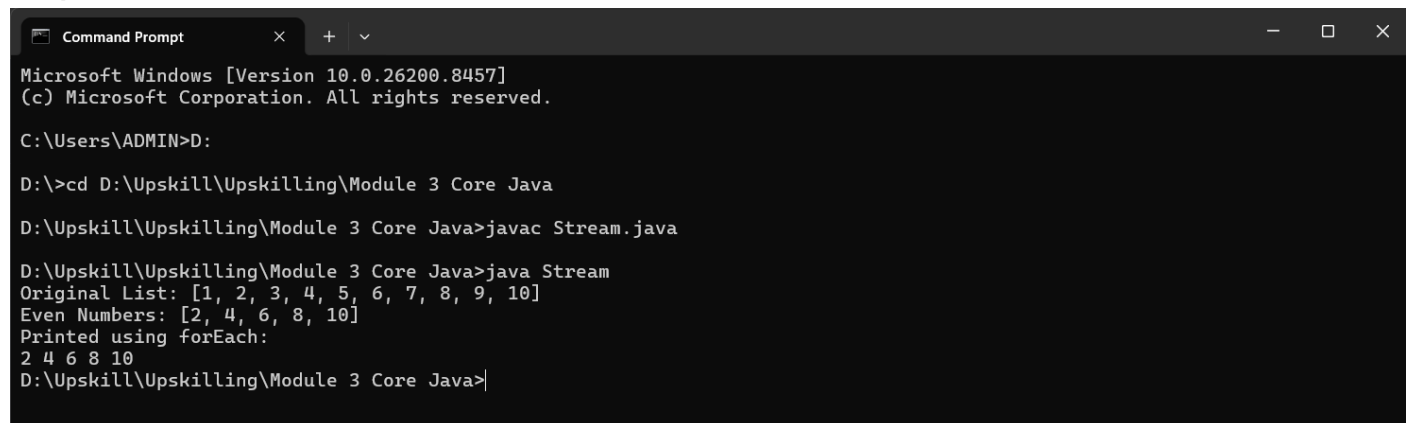
        System.out.println("Printed using forEach: ");

        numbers.stream().filter(n->n%2==0).forEach(n->System.out.print(n+" "));

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Stream.java

D:\Upskill\Upskilling\Module 3 Core Java>java Stream
Original List: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Even Numbers: [2, 4, 6, 8, 10]
Printed using forEach:
2 4 6 8 10
D:\Upskill\Upskilling\Module 3 Core Java>
```

29. Records

Objective: Use Java Records to create immutable and concise data-holding classes.

Code:

```
import java.util.*;

import java.util.stream.Collectors;

record Person(String name, int age){}

public class Records{

    public static void main(String[] args){

        Person p1=new Person("Mano",22);

        Person p2=new Person("Sai",20);

        System.out.println("Person 1: "+p1);

        System.out.println("Person 2: "+p2);

        System.out.println();

        List<Person> people=Arrays.asList(

            new Person("Alice",30),

            new Person("Bob",25),

            new Person("Charlie",35),

            new Person("Mano",17),

            new Person("Bharath",15)

        );

        System.out.println("List of People: "+people);

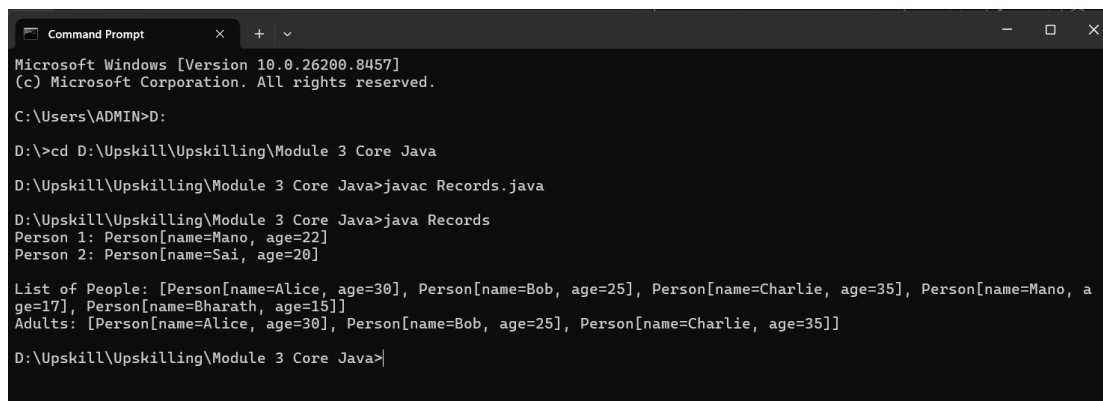
        List<Person> adults=people.stream().filter(p->p.age()>=18).collect(Collectors.toList());

        System.out.println("Adults: "+adults);

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java
D:\Upskill\Upskilling\Module 3 Core Java>javac Records.java
D:\Upskill\Upskilling\Module 3 Core Java>java Records
Person 1: Person[name=Mano, age=22]
Person 2: Person[name=Sai, age=20]

List of People: [Person[name=Alice, age=30], Person[name=Bob, age=25], Person[name=Charlie, age=35], Person[name=Mano, a
ge=17], Person[name=Bharath, age=15]]
Adults: [Person[name=Alice, age=30], Person[name=Bob, age=25], Person[name=Charlie, age=35]]

D:\Upskill\Upskilling\Module 3 Core Java>
```

30. Pattern Matching for Switch

Objective: Simplify conditional logic using enhanced switch expressions and pattern matching.

Code:

```
public class Pattern{

    public static void main(String[] args){

        checkType("Hello");

        checkType(123);

        checkType(3.14);

        checkType(true);

        checkType(null);

    }

    public static void checkType(Object obj){

        String ans=switch(obj){

            case String s->"It's a String: "+s;

            case Integer i->"It's an Integer: "+i;

            case Double d->"It's a Double: "+d;

            case Boolean b->"It's a Boolean: "+b;

            case null->"It's null";

            default->"Unknown type";

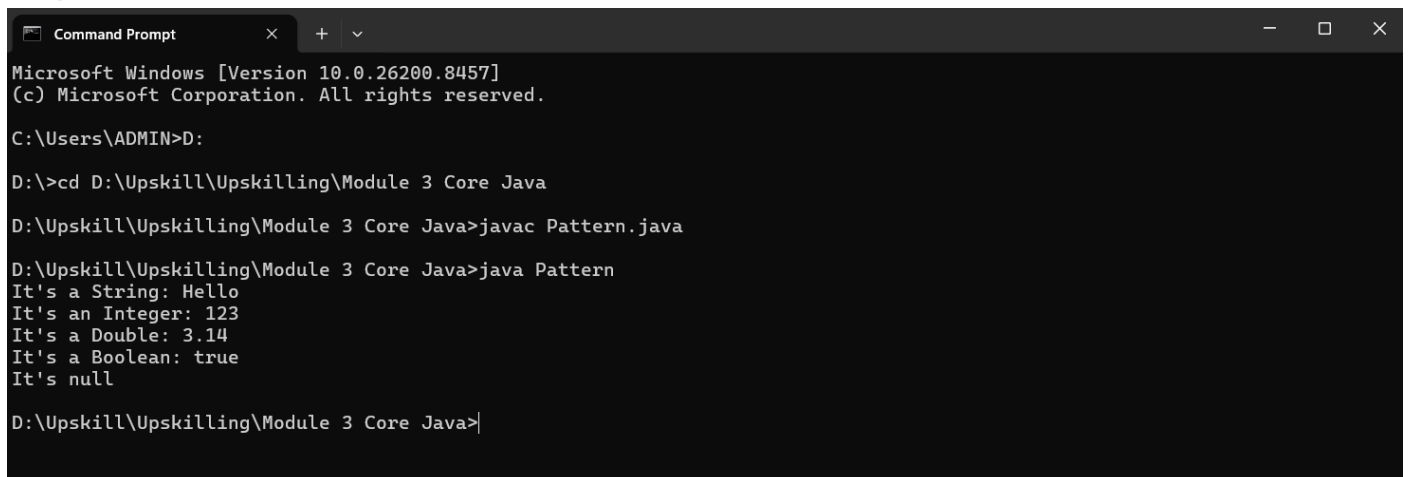
        };

        System.out.println(ans);

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Pattern.java

D:\Upskill\Upskilling\Module 3 Core Java>java Pattern
It's a String: Hello
It's an Integer: 123
It's a Double: 3.14
It's a Boolean: true
It's null

D:\Upskill\Upskilling\Module 3 Core Java>
```

31. Basic JDBC Connection

Objective: Connect Java applications with relational databases and retrieve data.

Code:

```
import java.sql.*;

public class JdbcConnection{

    private static final String URL="jdbc:mysql://localhost:3306/studentsdb";

    private static final String USER="root";

    private static final String PASSWORD="maX@89sql";

    public static void main(String[] args){

        try{Class.forName("com.mysql.cj.jdbc.Driver");

        } catch(ClassNotFoundException e){

            System.out.println("MySQL JDBC Driver not found");

            e.printStackTrace();

        }

        String query="Select *from students";

        try(Connection con=DriverManager.getConnection(URL,USER,PASSWORD);

        Statement stmt=con.createStatement();

        ResultSet rs=stmt.executeQuery(query)){

            System.out.println("Connected to the database successfully");

            System.out.printf("%5s | %-20s | %3s | %-20s\n", "ID", "Name", "Age", "Major");

            while(rs.next()){

                int id=rs.getInt("id");

                String name=rs.getString("name");

                int age=rs.getInt("age");

                String major=rs.getString("major");

                System.out.printf("%5d | %-20s | %3d | %-20s\n", id, name, age, major);

            }

        }catch(SQLException e){

            System.out.println("Database operation failed");

            e.printStackTrace();

        }

    }

}
```

Schema.sql

```
create database studentsdb;

use studentsdb;

create table students(

id int primary key auto_increment,

name varchar(50) not null,

age int,

major varchar(50)

);

insert into students(name,age,major) values

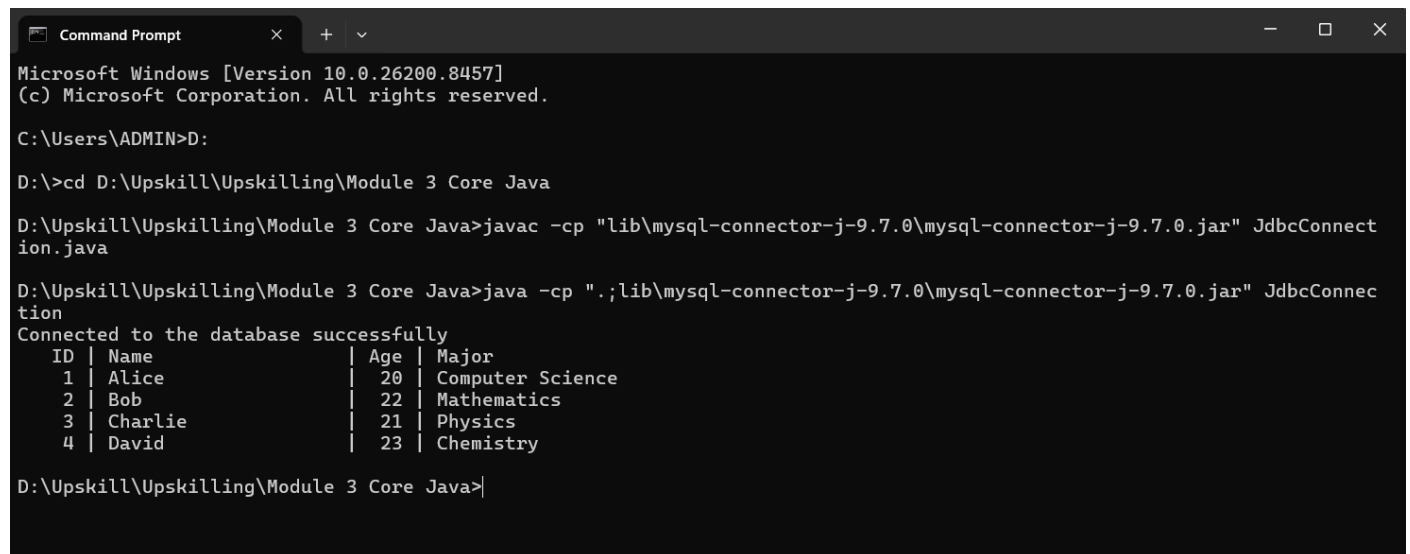
('Alice',20,'Computer Science'),

('Bob',22,'Mathematics'),

('Charlie',21,'Physics'),

('David',23,'Chemistry');
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac -cp "lib\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" JdbcConnection.java

D:\Upskill\Upskilling\Module 3 Core Java>java -cp ".;lib\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" JdbcConnection
Connected to the database successfully
  ID | Name      | Age | Major
  ---|---|---|---
  1  | Alice     | 20  | Computer Science
  2  | Bob       | 22  | Mathematics
  3  | Charlie   | 21  | Physics
  4  | David     | 23  | Chemistry

D:\Upskill\Upskilling\Module 3 Core Java>
```

32. Thread Creation

Objective: Perform database insert and update operations using JDBC.

Code:

```
public class Student{

    private int id;

    private String name;

    private int age;

    private String major;

    public Student(int id, String name, int age, String major){

        this.id=id;

        this.name=name;

        this.age=age;

        this.major=major;

    }

    public int getId(){return id;}

    public void setId(int id){this.id=id;}

    public String getName(){return name;}

    public void setName(String name){this.name=name;}

    public int getAge(){return age;}

    public void setAge(int age){this.age=age;}

    public String getMajor(){return major;}

    public void setMajor(String major){this.major=major;}

}

import java.sql.*;

public class StudentDAO{

    private static final String URL="jdbc:mysql://localhost:3306/studentsdb";

    private static final String USER="root";

    private static final String PASSWORD="maX@89sql";

    private Connection getConnection() throws SQLException{

        return DriverManager.getConnection(URL,USER,PASSWORD);

    }

    public boolean insertStudent(Student student){
```



```

String query="INSERT INTO students (id, name, age, major) VALUES (?, ?, ?, ?)";

try(Connection con=getConnection();

    PreparedStatement pstmt=con.prepareStatement(query)){

    pstmt.setInt(1, student.getId());

    pstmt.setString(2, student.getName());

    pstmt.setInt(3, student.getAge());

    pstmt.setString(4, student.getMajor());

    int rowsAffected=pstmt.executeUpdate();

    return rowsAffected>0;

}catch(SQLException e){

    System.out.println("Failed to insert student");

    e.printStackTrace();

    return false;

}

}

public boolean updateStudent(Student student){

    String query="UPDATE students SET name=?, age=?, major=? WHERE id=?";

    try(Connection con=getConnection();

        PreparedStatement pstmt=con.prepareStatement(query)){

        pstmt.setString(1, student.getName());

        pstmt.setInt(2, student.getAge());

        pstmt.setString(3, student.getMajor());

        pstmt.setInt(4, student.getId());

        int rowsAffected=pstmt.executeUpdate();

        return rowsAffected>0;

    }catch(SQLException e){

        System.out.println("Failed to update student");

        e.printStackTrace();

        return false;

    }

}

}

```

```
public class Main{

    public static void main(String[] args) {

        Student student = new Student(5, "John Doe", 20, "Computer Science");

        StudentDAO studentDAO = new StudentDAO();

        if(studentDAO.insertStudent(student)){

            System.out.println("Student inserted successfully");

        } else {

            System.out.println("Failed to insert student");

        }

        Student updatedStudent = new Student(5, "John Doe", 21, "Software Engineering");

        if(studentDAO.updateStudent(updatedStudent)){

            System.out.println("Student updated successfully");

        } else {

            System.out.println("Failed to update student");

        }

    }

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Student.java StudentDAO.java Main.java

D:\Upskill\Upskilling\Module 3 Core Java>java -cp ".;lib\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" Main
Student inserted successfully
Student updated successfully

D:\Upskill\Upskilling\Module 3 Core Java>
```

33. Transaction Handling in JDBC

Objective: Manage database transactions to ensure data consistency and integrity.

Code:

```
import java.sql.*;

public class Account{

    private static final String URL="jdbc:mysql://localhost:3306/studentsdb";

    private static final String USER="root";

    private static final String PASSWORD="maX@89sql";

    public static void main(String[] args){

        executeTransfer(1, 2, 200.00);

    }

    public static void executeTransfer(int fromAccountId, int toAccountId, double amount){

        String debitSql="UPDATE accounts SET balance = balance - ? WHERE account_id = ?";

        String creditSql="UPDATE accounts SET balance = balance + ? WHERE account_id = ?";

        Connection con=null;

        PreparedStatement debitStmt=null;

        PreparedStatement creditStmt=null;

        try{

            con=DriverManager.getConnection(URL, USER, PASSWORD);

            con.setAutoCommit(false);

            debitStmt=con.prepareStatement(debitSql);

            debitStmt.setDouble(1, amount);

            debitStmt.setInt(2, fromAccountId);

            int debitRows=debitStmt.executeUpdate();

            creditStmt=con.prepareStatement(creditSql);

            creditStmt.setDouble(1, amount);

            creditStmt.setInt(2, toAccountId);

            int creditRows=creditStmt.executeUpdate();

            if(debitRows==1 && creditRows==1){

                con.commit();

                System.out.println("Transfer successful! $" + amount + " transferred from account " + fromAccountId + " to account " + toAccountId);

            }else{
```

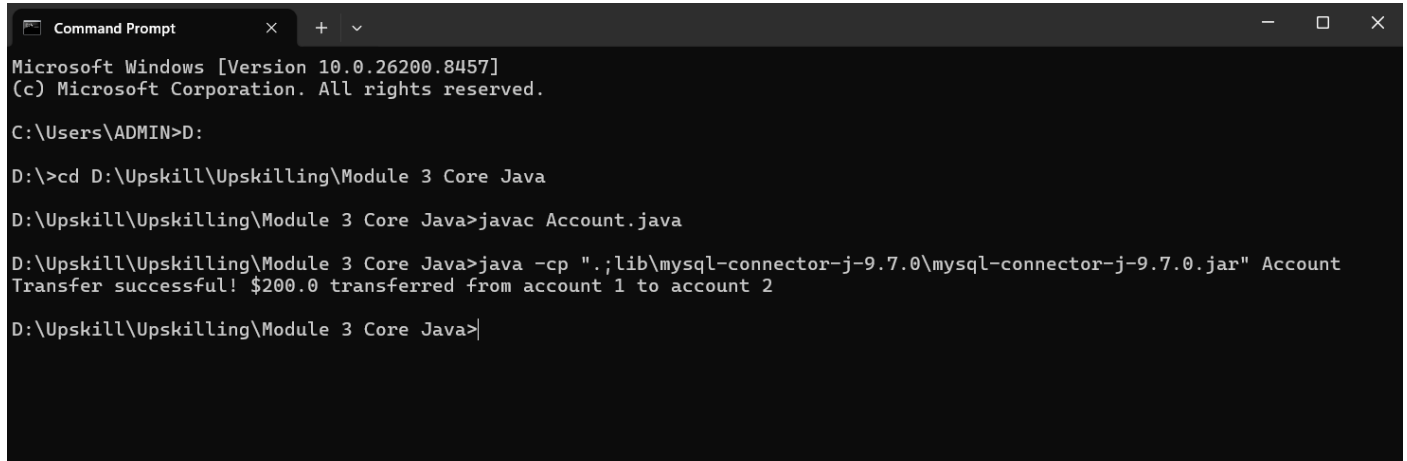
```
        con.rollback();

        System.out.println("Transfer failed: Account not found or insufficient funds");
    }
} catch (SQLException e) {
    e.printStackTrace();

    System.err.println("Critical Exception occurred during transfer. Rolling back transaction immediately.");

    try {
        if (con != null) {
            con.rollback();
        }
    } catch (SQLException ex) {
        System.err.println("Error occurred while rolling back transaction.");
        ex.printStackTrace();
    }
} finally {
    try {
        if (debitStmt != null) debitStmt.close();
        if (creditStmt != null) creditStmt.close();
        if (con != null) {
            con.setAutoCommit(true);
            con.close();
        }
    } catch (SQLException e) {
        System.err.println("Error occurred while closing resources.");
        e.printStackTrace();
    }
}
}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Account.java

D:\Upskill\Upskilling\Module 3 Core Java>java -cp ".;lib\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" Account
Transfer successful! $200.0 transferred from account 1 to account 2

D:\Upskill\Upskilling\Module 3 Core Java>
```

34. Create and Use Java Modules

Objective: Understand Java's module system and modular application development.

Code:

// src/com/Utils/module-info.java

```
module com.Utils {  
    exports com.Utils;  
}
```

// src/com/Utils/com/Utils/StringFormatter.java

```
package com.Utils;  
  
public class StringFormatter {  
    public static String toBanner(String message) {  
        return "=== " + message.toUpperCase() + " ===";  
    }  
}
```

// src/com.greetings/module-info.java

```
module com.greetings {  
    requires com.Utils;  
}
```

// src/com.greetings/com/greetings/Main.java

```
package com.greetings;  
  
import com.Utils.StringFormatter;  
  
public class Main {  
    public static void main(String[] args) {  
        String welcomeMessage = "Hello from the modular world!";  
        String formattedOutput = StringFormatter.toBanner(welcomeMessage);  
        System.out.println(formattedOutput);  
    }  
}
```

Output:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Account.java

D:\Upskill\Upskilling\Module 3 Core Java>java -cp ".;lib\mysql-connector-j-9.7.0\mysql-connector-j-9.7.0.jar" Account
Transfer successful! $200.0 transferred from account 1 to account 2

D:\Upskill\Upskilling\Module 3 Core Java>cd ProjectRoot

D:\Upskill\Upskilling\Module 3 Core Java\ProjectRoot>javac -d mods/com.utils src/com/utils/module-info.java src/com/utl
s/com/utils/StringFormatter.java

D:\Upskill\Upskilling\Module 3 Core Java\ProjectRoot>javac --module-path mods -d mods/com.greetings src/com.greetings/mo
dule-info.java src/com.greetings/com/greetings/Main.java

D:\Upskill\Upskilling\Module 3 Core Java\ProjectRoot>java --module-path mods -m com.greetings/com.greetings.Main
=== HELLO FROM THE MODULAR WORLD! ===

D:\Upskill\Upskilling\Module 3 Core Java\ProjectRoot>
```

35. TCP Client-Server Chat

Objective: Implement network communication using Java sockets.

Code:

// ChatServer.java

```
import java.io.*;

import java.net.*;

public class ChatServer{

    public static void main(String[] args){

        int port=5000;

        System.out.println("Starting chat server on port "+port);

        try(ServerSocket serverSocket=new ServerSocket(port);

            Socket clientSocket=serverSocket.accept();

            BufferedReader in=new BufferedReader(new InputStreamReader(clientSocket.getInputStream()));

            PrintWriter out=new PrintWriter(clientSocket.getOutputStream(), true);

            BufferedReader consoleInput=new BufferedReader(new InputStreamReader(System.in))){

                System.out.println("Client connected! Start chatting. Type 'bye' to exit.");

                String clientMessage;

                String serverResponse;

                while(true){

                    clientMessage = in.readLine();

                    if(clientMessage == null){

                        System.out.println("Client closed the connection.");

                        break;

                    }

                    if(clientMessage.equalsIgnoreCase("bye")){

                        System.out.println("Client requested to end chat.");

                        break;

                    }

                    System.out.println("Client: "+clientMessage);

                    System.out.print("Server (You): ");

                    serverResponse = consoleInput.readLine();

                    if(serverResponse == null){

                        System.out.println("Console input closed. Shutting down server.");
```



```

        break;
    }
    out.println(serverResponse);
    if(serverResponse.equalsIgnoreCase("bye")){
        System.out.println("Closing Server connection...");
        break;
    }
}
}
}catch(IOException e){
    System.err.println("Server exception encountered.");
    e.printStackTrace();
}
}
}

```

// ChatClient.java

```

import java.io.*;
import java.net.*;

public class ChatClient{

    public static void main(String[] args){

        String host="localhost";

        int port=5000;

        System.out.println("Connecting to Server at " + host + ":" + port);

        try(Socket socket=new Socket(host,port);

            BufferedReader in=new BufferedReader(new InputStreamReader(socket.getInputStream()));

            PrintWriter out=new PrintWriter(socket.getOutputStream(), true);

            BufferedReader consoleInput=new BufferedReader(new InputStreamReader(System.in))){

            System.out.println("Connected successfully! Type your first message. Type 'bye' to exit.");

            String clientMessage;

            String serverResponse;

            while(true){

                System.out.print("Client (You): ");

                // read message from console and send to server

```

```
        clientMessage = consoleInput.readLine();

        if(clientMessage == null){

            System.out.println("Console closed. Exiting.");

            break;

        }

        out.println(clientMessage);

        if(clientMessage.equalsIgnoreCase("bye")){

            System.out.println("Closing client connection.");

            break;

        }

        // read server response

        serverResponse = in.readLine();

        if(serverResponse == null){

            System.out.println("Server closed the connection.");

            break;

        }

        System.out.println("Server: " + serverResponse);

    }

} catch(IOException e){

    System.err.println("Client exception encountered.");

    e.printStackTrace();

}

}
```

Output:

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac ChatServer.java

D:\Upskill\Upskilling\Module 3 Core Java>java ChatServer
Starting chat server on port 5000
Client connected! Start chatting. Type 'bye' to exit.
Client: Hi
Server (You): Hello
Client requested to end chat.

D:\Upskill\Upskilling\Module 3 Core Java>
```

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac ChatClient.java

D:\Upskill\Upskilling\Module 3 Core Java>java ChatClient
Connecting to Server at localhost:5000
Connected successfully! Type your first message. Type 'bye' to exit.
Client (You): Hi
Server: Hello
Client (You): Bye
Closing client connection.

D:\Upskill\Upskilling\Module 3 Core Java>
```

36. HTTP Client API

Objective: Make HTTP requests and consume web APIs from Java applications.

Code:

```
import java.net.*;

import java.net.http.HttpClient;

import java.net.http.HttpRequest;

import java.net.http.HttpResponse;

import java.io.IOException;

import java.time.Duration;

public class GitHubAPI{

    public static void main(String[] args){

        String url="https://github.com";

        HttpClient client=HttpClient.newBuilder()

            .connectTimeout(Duration.ofSeconds(10))

            .followRedirects(HttpClient.Redirect.NORMAL)

            .build();

        HttpRequest request=HttpRequest.newBuilder()

            .uri(URI.create(url))

            .timeout(Duration.ofSeconds(10))

            .header("Accept","application/json")

            .header("User-Agent","Java-11-HttpClient-App")

            .GET().build();

        System.out.println("Sending GET request to: "+url+"...");

        try{

            HttpResponse<String> res=client.send(request,HttpResponse.BodyHandlers.ofString());

            System.out.println("\n--- HTTP Response Data ---");

            System.out.println("Status Code: "+res.statusCode());

            if(res.statusCode()==200){

                System.out.println("Success! Content Body snippet:");

                String body=res.body();

                System.out.println(body.length()>200?body.substring(0,500)+" ...":body);

            }else{

                System.err.println("Failed Connection! Server responded with message:\n" + res.body());

            }

        }

    }

}
```

```

    }

} catch (IOException e) {

    System.out.println("Network I/O error occurred during transmission.");

    e.printStackTrace();

} catch (InterruptedException e) {

    System.err.println("The operational execution thread was interrupted.");

    Thread.currentThread().interrupt();

}

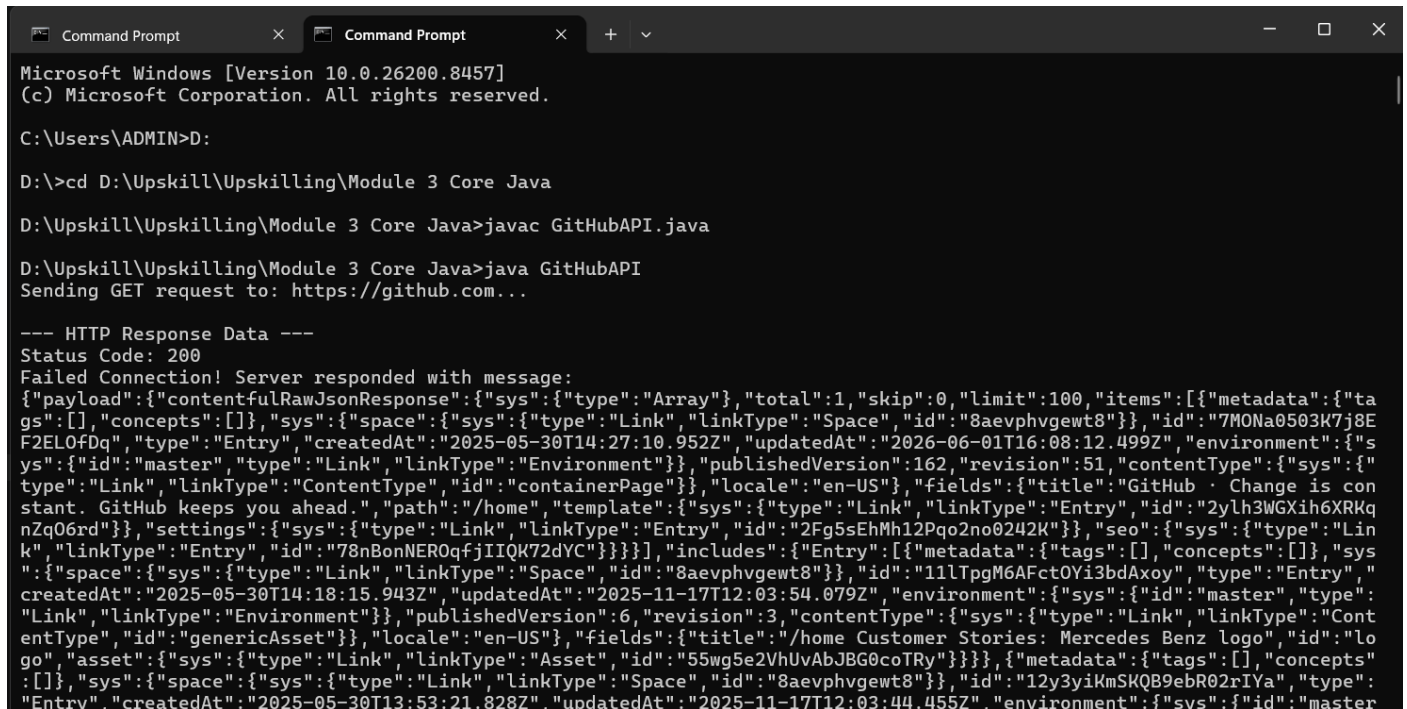
}

}

}

```

Output:



```

Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac GitHubAPI.java

D:\Upskill\Upskilling\Module 3 Core Java>java GitHubAPI
Sending GET request to: https://github.com...

--- HTTP Response Data ---
Status Code: 200
Failed Connection! Server responded with message:
{"payload":{"contentfulRawJsonResponse":{"sys":{"type":"Array"},"total":1,"skip":0,"limit":100,"items":[{"metadata":{"tags":[""],"concepts":[""],"sys":{"space":{"sys":{"type":"Link","linkType":"Space","id":"8aevphvgewt8"},"id":"7MONa0503K7j8EF2ELOfDq","type":"Entry","createdAt":"2025-05-30T14:27:10.952Z","updatedAt":"2026-06-01T16:08:12.499Z","environment":{"sys":{"id":"master","type":"Link","linkType":"Environment"},"publishedVersion":162,"revision":51,"contentType":{"sys":{"type":"Link","linkType":"ContentType","id":"containerPage"},"locale":"en-US"},"fields":{"title":"GitHub · Change is constant. GitHub keeps you ahead.","path":"/home","template":{"sys":{"type":"Link","linkType":"Entry","id":"2ylh3WGXih6XRRqnZq06rd"},"settings":{"sys":{"type":"Link","linkType":"Entry","id":"2Fg5sEhMh12Pqo2no0242K"},"seo":{"sys":{"type":"Link","linkType":"Entry","id":"78nBonNEROqfjIIQK72dYC"}}}],includes":{"Entry":[{"metadata":{"tags":[""],"concepts":[""],"sys":{"space":{"sys":{"type":"Link","linkType":"Space","id":"8aevphvgewt8"},"id":"11lTpgM6AFct0Yi3bdAxoy","type":"Entry","createdAt":"2025-05-30T14:18:15.943Z","updatedAt":"2025-11-17T12:03:54.079Z","environment":{"sys":{"id":"master","type":"Link","linkType":"Environment"},"publishedVersion":6,"revision":3,"contentType":{"sys":{"type":"Link","linkType":"ContentType","id":"genericAsset"},"locale":"en-US"},"fields":{"title":"/home Customer Stories: Mercedes Benz logo","id":"logo","asset":{"sys":{"type":"Link","linkType":"Asset","id":"55wg5e2VhUvAbJBG0coTRy"}}},"metadata":{"tags":[""],"concepts":[""],"sys":{"space":{"sys":{"type":"Link","linkType":"Space","id":"8aevphvgewt8"},"id":"12y3yiKmSKQB9ebR02rIYa","type":"Entry","createdAt":"2025-05-30T13:53:21.828Z","updatedAt":"2025-11-17T12:03:44.455Z","environment":{"sys":{"id":"master

```

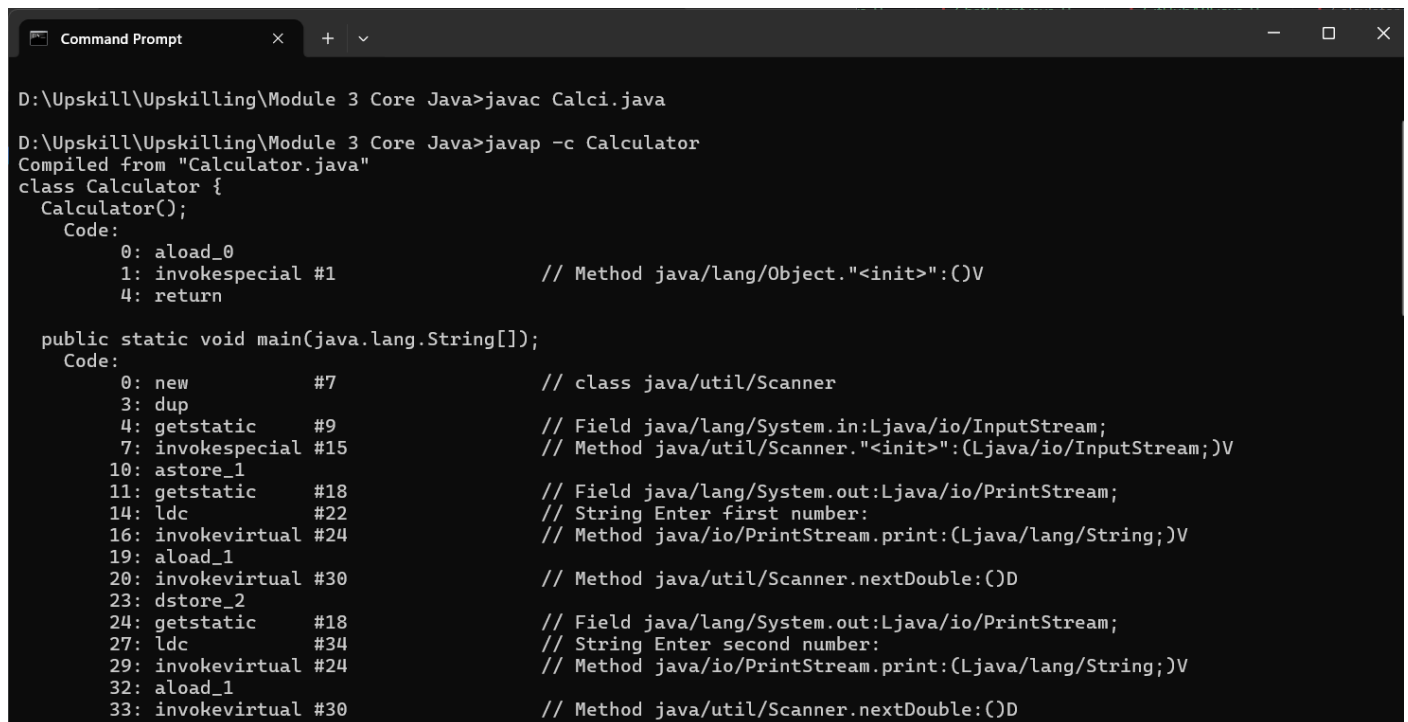
37. Using javap to Inspect Bytecode

Objective: Explore compiled bytecode and understand Java's execution process.

Code:

```
public class Calci{  
  
    public int add(int a,int b){  
  
        return a+b;  
  
    }  
  
}
```

Output:



```
D:\Upskill\Upskilling\Module 3 Core Java>javac Calci.java  
  
D:\Upskill\Upskilling\Module 3 Core Java>javap -c Calculator  
Compiled from "Calculator.java"  
class Calculator {  
    Calculator();  
    Code:  
        0: aload_0  
        1: invokespecial #1          // Method java/lang/Object."<init>":()V  
        4: return  
  
    public static void main(java.lang.String[]);  
    Code:  
        0: new           #7          // class java/util/Scanner  
        3: dup  
        4: getstatic    #9          // Field java/lang/System.in:Ljava/io/InputStream;  
        7: invokespecial #15         // Method java/util/Scanner."<init>":(Ljava/io/InputStream;)V  
       10: astore_1  
       11: getstatic    #18         // Field java/lang/System.out:Ljava/io/PrintStream;  
       14: ldc         #22          // String Enter first number:  
       16: invokevirtual #24         // Method java/io/PrintStream.print:(Ljava/lang/String;)V  
       19: aload_1  
       20: invokevirtual #30         // Method java/util/Scanner.nextDouble:()D  
       23: dstore_2  
       24: getstatic    #18         // Field java/lang/System.out:Ljava/io/PrintStream;  
       27: ldc         #34          // String Enter second number:  
       29: invokevirtual #24         // Method java/io/PrintStream.print:(Ljava/lang/String;)V  
       32: aload_1  
       33: invokevirtual #30         // Method java/util/Scanner.nextDouble:()D
```

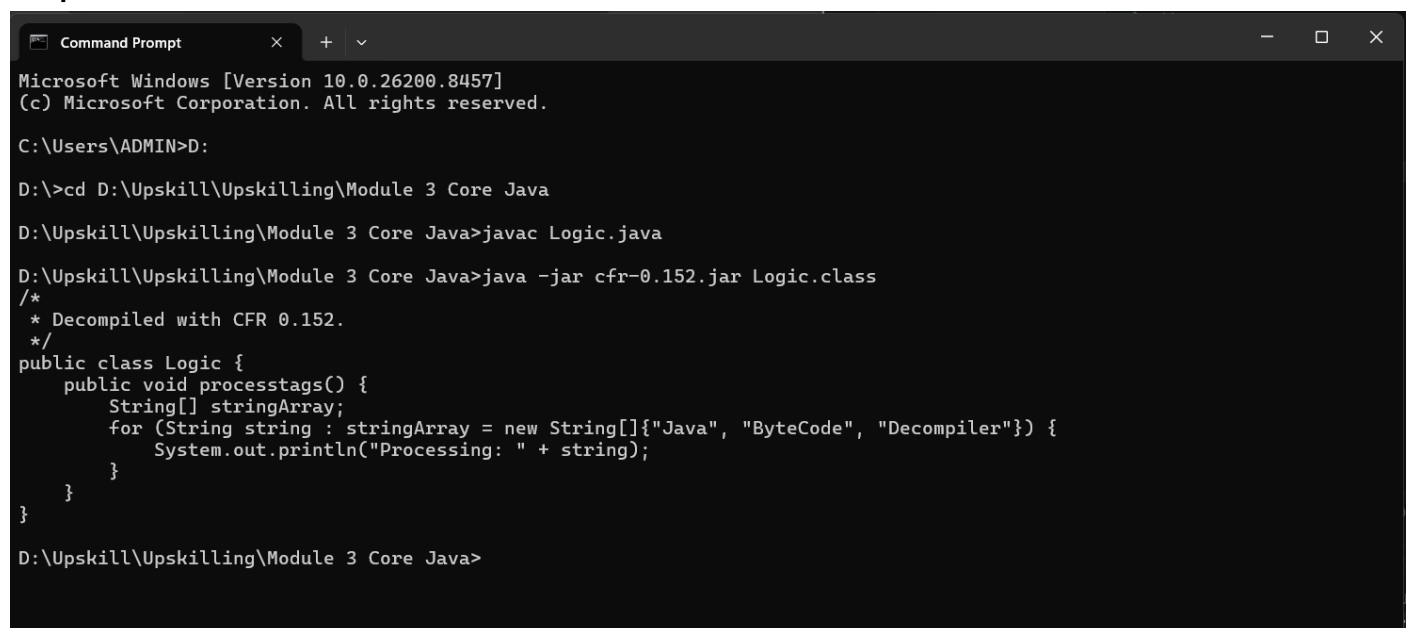
38. Decompile a Class File

Objective: Reverse engineer compiled Java classes and analyze generated source code.

Code:

```
public class Logic{  
    public void processtags(){  
        String[] tags={"Java","ByteCode","Decompiler"};  
        for(String t:tags){  
            System.out.println("Processing: "+t);  
        }  
    }  
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>cd D:\Upskill\Upskilling\Module 3 Core Java  
  
D:\Upskill\Upskilling\Module 3 Core Java>javac Logic.java  
  
D:\Upskill\Upskilling\Module 3 Core Java>java -jar cfr-0.152.jar Logic.class  
/*  
 * Decompiled with CFR 0.152.  
 */  
public class Logic {  
    public void processtags() {  
        String[] stringArray;  
        for (String string : stringArray = new String[]{"Java", "ByteCode", "Decompiler"}) {  
            System.out.println("Processing: " + string);  
        }  
    }  
}
```

39. Reflection in Java

Objective: Use the Reflection API to inspect and invoke classes, methods, and fields dynamically.

Code:

```
import java.lang.reflect.Method;

import java.lang.reflect.Parameter;

public class Reflection{

    public static class Calculator{

        public void add(int a,int b){

            System.out.println(" ->Execution Result: "+(a+b));

        }

        private void greet(String name){

            System.out.println(" -> Execution Result: Hello, "+name+"!");

        }

    }

    public static void main(String[] args){

        try{

            Class<?> claz=Class.forName("Reflection$Calculator");

            System.out.println("Successfully loaded class: \" + claz.getName() + \"\\n");

            Object cal=claz.getDeclaredConstructor().newInstance();

            Method[] methods=claz.getDeclaredMethods();

            for(Method m:methods){

                System.out.println("Method Name: "+m.getName());

                System.out.print("Parameters: ");

                Parameter[] params=m.getParameters();

                if(params.length==0) System.out.println("None");

                else{

                    for(int i=0;i<params.length;i++){

                        System.out.print(params[i].getType().getSimpleName()+" "+params[i].getName());

                        if(i<params.length-1) System.out.print(", ");

                    }

                    System.out.println();

                }

                if(m.getName().equals("add")){
```



```
        m.invoke(cal,10,20);

    }else if(m.getName().equals("greet")){

        m.setAccessible(true);

        m.invoke(cal,"Vijay");

    }

}

}

}

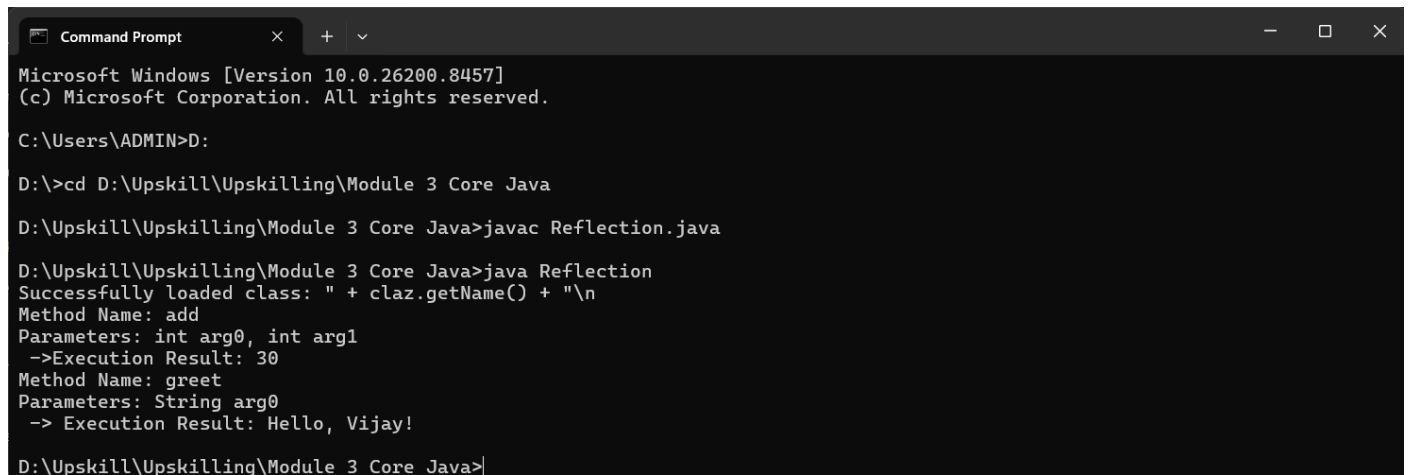
}

}

}

}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Reflection.java

D:\Upskill\Upskilling\Module 3 Core Java>java Reflection
Successfully loaded class: " + clazz.getName() + "\n
Method Name: add
Parameters: int arg0, int arg1
->Execution Result: 30
Method Name: greet
Parameters: String arg0
-> Execution Result: Hello, Vijay!

D:\Upskill\Upskilling\Module 3 Core Java>
```

40. Virtual Threads (Java 21)

Objective: Use lightweight virtual threads to build scalable concurrent applications.

Code:

```
import java.time.Duration;

import java.util.concurrent.CountDownLatch;

public class VirtualThread{

    private static final int tasks=100000;

    public static void main(String[] args) throws InterruptedException{

        long s=System.currentTimeMillis();

        CountDownLatch lat1=new CountDownLatch(tasks);

        for(int i=0;i<tasks;i++){

            int id=i;

            Thread.startVirtualThread(()->{

                if(id%50000==0) System.out.println("Virtual thread message " + id);

                lat1.countDown();

            });

        }

        lat1.await();

        System.out.println("Virtual Thread Time: "+(System.currentTimeMillis()-s)+"ms\n");

        s=System.currentTimeMillis();

        CountDownLatch lat2=new CountDownLatch(tasks);

        try{

            for(int i=0;i<tasks;i++){

                int id=i;

                Thread.ofPlatform().start(()->{

                    if(id%50000==0) System.out.println("Platform thread message " + id);

                    lat2.countDown();

                });

            }

            lat2.await();

            System.out.println("Platform Threads Time: "+(System.currentTimeMillis()-s)+"ms");

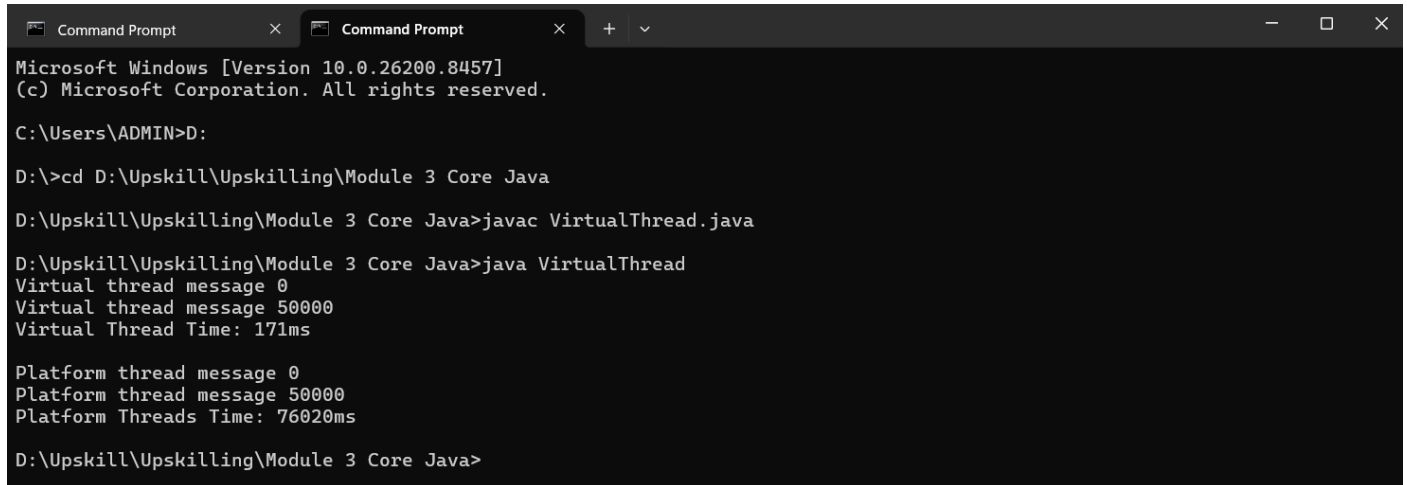
        }catch(OutOfMemoryError e){

            System.out.println("Platform threads crashed: Out of memory!");

        }
```

```
}  
  
}  
  
}
```

Output:



```
Microsoft Windows [Version 10.0.26200.8457]  
(c) Microsoft Corporation. All rights reserved.  
  
C:\Users\ADMIN>D:  
  
D:\>cd D:\Upskill\Upskilling\Module 3 Core Java  
  
D:\Upskill\Upskilling\Module 3 Core Java>javac VirtualThread.java  
  
D:\Upskill\Upskilling\Module 3 Core Java>java VirtualThread  
Virtual thread message 0  
Virtual thread message 50000  
Virtual Thread Time: 171ms  
  
Platform thread message 0  
Platform thread message 50000  
Platform Threads Time: 76020ms  
  
D:\Upskill\Upskilling\Module 3 Core Java>
```

41. Executor Service and Callable

Objective: Utilize Java concurrency utilities to execute tasks asynchronously and retrieve results.

Code:

```
import java.util.*;

import java.util.concurrent.Callable;

import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;

import java.util.concurrent.Future;

public class Executor{

    public static void main(String[] args) throws Exception{

        ExecutorService exe=Executors.newFixedThreadPool(3);

        List<Future<String>> future=new ArrayList<>();

        System.out.println("Submitting tasks");

        for(int i=1;i<=5;i++){

            final int taskId=i;

            Callable<String> task=()->{

                Thread.sleep(100);

                return "Result from task "+taskId+"(Executed by " + Thread.currentThread().getName() + ")";

            };

            future.add(exe.submit(task));

        }

        System.out.println("\nCollecting results (blocking via Future.get())...");

        for(Future<String> f:future){

            String res=f.get();

            System.out.println(res);

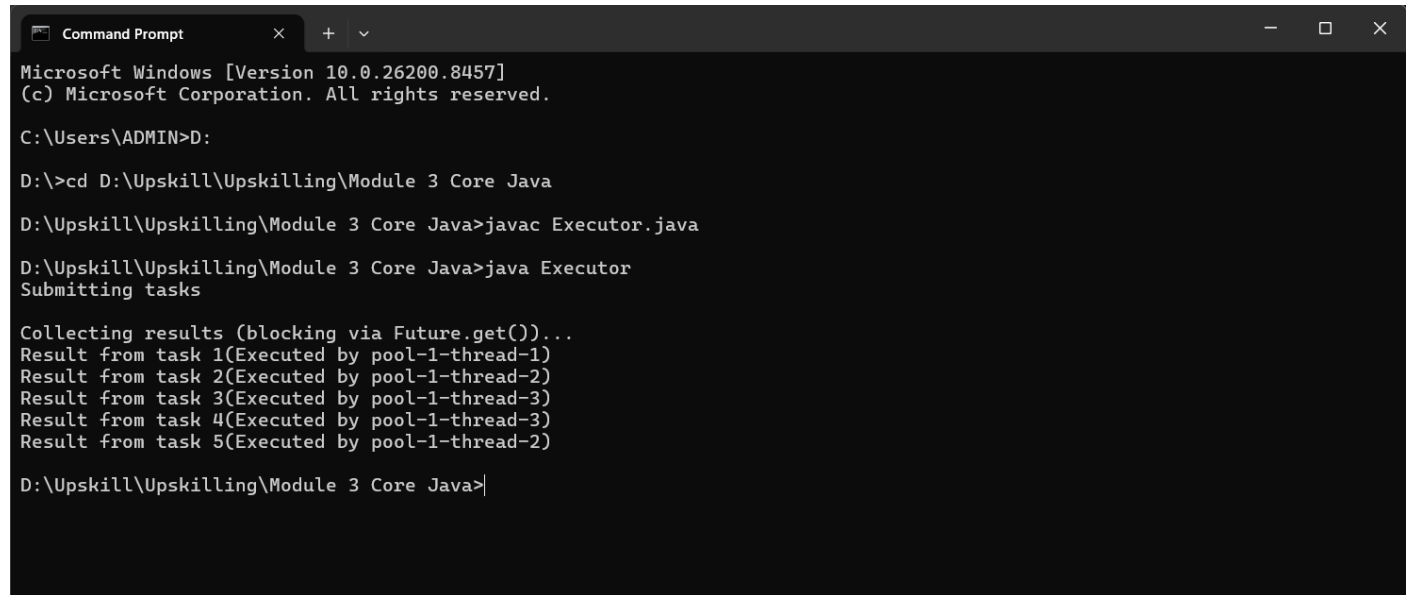
        }

        exe.shutdown();

    }

}
```

Output:



```
Command Prompt
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ADMIN>D:

D:\>cd D:\Upskill\Upskilling\Module 3 Core Java

D:\Upskill\Upskilling\Module 3 Core Java>javac Executor.java

D:\Upskill\Upskilling\Module 3 Core Java>java Executor
Submitting tasks

Collecting results (blocking via Future.get())...
Result from task 1(Executed by pool-1-thread-1)
Result from task 2(Executed by pool-1-thread-2)
Result from task 3(Executed by pool-1-thread-3)
Result from task 4(Executed by pool-1-thread-3)
Result from task 5(Executed by pool-1-thread-2)

D:\Upskill\Upskilling\Module 3 Core Java>
```